

Marine Plastic Debris Detection using Machine Learning

A major project report submitted in partial fulfillment of the requirement
for the award of degree of

Bachelor of Technology
in
Computer Science & Engineering

Submitted by
Harmandeep Singh Maan (221030090)

Under the guidance & supervision of
Mr. Sandeep Kumar Patel



**Department of Computer Science & Engineering and
Information Technology**
**Jaypee University of Information Technology, Wagnaghat, Solan -
173234 (India)**
MAY 2026

Candidate's Declaration

I hereby declare that the work presented in this major project report entitled '**Marine Plastic Debris Detection using Machine Learning**', submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering, in the Department of Computer Science & Engineering and Information Technology, Jaypee University of Information Technology, Waknaghat, is an authentic record of my own work carried out during the period from July 2025 to May 2026 under the supervision of Mr. Sandeep Kumar Patel. I further declare that the matter embodied in this report has not been submitted for the award of any other degree or diploma at any other university or institution.

Name & Sign: Harmandeep

Singh Maan Roll No.:

221030090

Date: 7th May 2026

This is to certify that the above statement made by the candidate is true to the best of my knowledge. Date: 7th May 2026

Place: JUIT

Supervisor Name & Sign: Mr. Sandeep Kumar

Patel Designation: Assistant Professor (Grade I)

Department: Computer Science & Engineering and Information Technology

Supervisor's Certificate

This is to certify that the major project report entitled '**Marine Plastic Debris Detection using Machine Learning**', submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering, in the Department of Computer Science & Engineering and Information Technology, Jaypee University of Information Technology, Waknaghat, is a bonafide project work carried out under my supervision during the period from July 2025 to May 2026. I have personally supervised the research work and confirm that it meets the standards required for submission. The project work has been conducted in accordance with ethical guidelines, and the matter embodied in the report has not been submitted elsewhere for the award of any other degree or diploma.

Date: 7th May 2026
Patel

Supervisor Name & Sign: Mr. Sandeep Kumar

Place: JUIT

Designation: Assistant Professor (Grade I)

Department: Computer Science &
Engineering and Information
Technology

ACKNOWLEDGEMENT

Firstly, we express my heartiest thanks and gratefulness to almighty God for His divine blessing makes it possible for us to complete the project work successfully.

I am really grateful and wish my profound indebtedness to Supervisor **Mr. Sandeep Kumar Patel**, Assistant Professor Grade I, Department of CSE Jaypee University of Information Technology, Wakhnaghat. Deep Knowledge & keen interest of my supervisor in the field of “**Machine Learning**” to carry out this project. His endless patience, scholarly guidance, continual encouragement, constant and energetic supervision, constructive criticism, valuable advice, reading many inferior drafts and correcting them at all stages have made it possible to complete this project.

I would like to express my heartiest gratitude to Dr. Sandeep Kumar Patel, Department of CSE, for his kind help to finish my project.

I would also generously welcome each one of those individuals who have helped me straightforwardly or in a roundabout way in making this project a win. In this unique situation, we might want to thank the various staff individuals, both educating and non-instructing, which have developed their convenient help and facilitated my undertaking.

Finally, I must acknowledge with due respect the constant support and patients of my parents.

Harmandeep Singh Maan (221030090)

TABLE OF CONTENT

Title	Page No:
List of tables	II
List of Figures	III
List of Abbreviation, Symbols or Nomenclature	IV
Abstract	V
Chapter-1 Introduction 1.1 Introduction 1.2 Problem Statement 1.3 Objectives 1.4 Significance and Motivation of the Project Work 1.5 Organization of the report	1-8
Chapter-2 Literature Review 2.1 Overview of Relevant Literature 2.2 Key Gaps in the Literature	9-15
Chapter-3 System Development 3.1 Requirements and Analysis 3.2 Project Design and Architecture 3.3 Data Preparation 3.4 Implementation 3.5 Key Challenges	16-49
Chapter-4 Testing 4.1 Testing Strategy 4.2 Test Cases and Outcomes	50-59
Chapter-5 Results and Evaluation 5.1 Results 5.2 Comparison with Existing Solutions	60-70
Chapter- 6 Conclusion and Future Scope 6.1 Conclusion 6.2 Future Scope	71-74
References	75-76

LIST OF TABLES

S No:	Title	Page No:
Table 2.1	Literature Review	8-14
Table 3.1	Resources Requirements	15
Table 5.1	Model Performance Comparison Across Training Runs	30
Table 5.2	Inference Results for Both Test Regions	31
Table 5.3	Comparison with Existing Solutions	34

LIST OF FIGURES

S.No.	Title	Page No.
Figure 3.1	Flowchart of Project Design and Architecture	13
Figure 3.2	Work Flow of Project Design and Architecture	14
Figure 3.3	Unet++ Architecture	16
Figure 3.4	Attention Mechanism	17
Figure 3.5	Preprocessing	18
Figure 3.6	Training Loop	20
Figure 3.7	Loss Function	20
Figure 3.8	Evaluation Pipeline	21
Figure 3.9	DebrisNet Dashboard - Idle State	24
Figure 3.10	DebrisNet Dashboard - Detection Results	24
Figure 4.1	Model Test Results - UNet++ Final Training Run	26
Figure 4.2	Motagua River Inference Result - 3 Panel View	27
Figure 4.3	South Africa Durban Inference Result	28
Figure 4.4	DebrisNet Dashboard - Motagua Detection	29
Figure 4.5	DebrisNet Dashboard - South Africa Detection	29
Figure 5.1	Model Performance Comparison Chart	31
Figure 5.2	Motagua River Debris Detection Result	32
Figure 5.3	DebrisNet Dashboard Motagua Result	32
Figure 5.4	South Africa Durban Dashboard Result	33
Figure 5.5	DebrisNet Dashboard Full View with Stats Bar	33

LIST OF ABBREVIATIONS, SYMBOLS OR NOMENCLATURE

Abbreviations	Meaning
AI	Artificial Intelligence
CNN	Convolutional Neural Network
SVM	Support Vector Machine
MARIDA	Marine Debris Archive
IoT	Internet of Things
GPU	Graphics Processing Unit
U-NET	A specific type of Convolutional Neural Network for image segmentation
JPEG	Joint Photographic Experts Group
ML	Machine Learning
YOLO	You Only Look Once (A real-time object detection model)
ISPRS	International Society for Photogrammetry and Remote Sensing
GIS	Geographic Information System
GDAL	Geospatial Data Abstraction Library
API	Application Programming Interface
ESA	European Space Agency

ABSTRACT

Marine plastic pollution has become one of the most critical environmental problems of the 21st century. Every year millions of tonnes of plastic debris find their way into our oceans, threatening marine ecosystems, fisheries and even human health. Ship-based surveys and other traditional monitoring methods are expensive, time-consuming, and spatially limited. Recent advances in satellite remote sensing and machine learning have created opportunities for large scale, automated monitoring of marine debris. this project brings scalable ai solutions to the fight against marine plastic pollution for environmental action.

We present here a machine-learning pipeline to detect, segment and map floating marine plastic debris using multispectral satellite imagery acquired by the Sentinel-2 satellite and the MARIDA dataset. The system combines preprocessing such as atmospheric correction, band resampling and data augmentation with a UNet++ deep learning segmentation model, trained with imbalance-aware training strategies with weighted loss functions. The model was trained for 50 epochs using PyTorch Lightning on a CPU system with an AMD Ryzen 5 5625U processor and 15.3 GB RAM. The final test accuracy was 99.90%, F1 Score was 14.74% and IoU was 7.96%.

We inferred two real Sentinel-2 scenes, Motagua River Plume in Guatemala (2020-09-18) and Durban, South Africa (2019-04-29). In the Motagua River scene, 2,712 detected debris pixels created a distinct river plume debris trail. The South Africa scene had 56 detected pixels in the port area. We developed a professional web-based dashboard named DebrisNet using HTML, CSS, and JavaScript to display the detection results interactively. Outputs include georeferenced probability masks, binary debris maps and an interactive visualisation dashboard.

The solution we developed can be useful for NGOs, policy makers and researchers to monitor zones of plastic accumulation and to guide sustainable marine policies. This project will connect research and action in environment and will contribute to the fight against marine plastic pollution through scalable AI-driven solutions.

.

CHAPTER 01: INTRODUCTION

1.1 INTRODUCTION

The oceans cover approximately 70% of the Earth and are considered an integral part of the Earth's environment. They manage the global atmosphere, sequester millions of tons of CO₂, and create much of the oxygen that oceans and terrestrial organisms need to survive. The oceans serve an essential role in supporting living organisms and regulating global climate. In addition to supporting life as we know it, the oceans are home to an extraordinary diversity of organisms and provide a foundation for many people's livelihoods. The oceans support fisheries, tourism, international trade, and transportation; they also support the maintenance of both natural ecosystems and global economies. Therefore, the oceans can be viewed as the Earth's largest and most important common resource.

But one of our longest-term inventions of mankind plastic pollution is causing even more threats to this important life-supporting system. Millions of tonnes of plastic waste from things like consumer packaging, single-use products, and industrial waste are dumped into the oceans every year, and unlike organic materials, plastic doesn't break down naturally. Depending on the type of plastic and the conditions of the environment, it can take a hundred years or more for plastics to break down (due to their being made chemically). However, as they are gradually breaking down, they can still exist in the ocean as very small particles (micro-plastics) of the original material, which can create enormous floating "garbage patches" in the ocean gyres, entrap marine animals, choke marine animals when they ingest them, and eliminate the natural habitats of marine animals.

The process of degradation is producing microplastic particles; which are less than five millimetres in diameter, and these create an invisible but very real risk for humans. Microplastics are being found wherever there is life; plankton, fish, shellfish, and eventually to humans by food chain transfer. Research into human health impacts of microplastics is still in early stages but early results suggest that microplastics are likely to contain harmful chemicals that will cause biological impairment, or may present long-term health problems. Marine plastic pollution is therefore not just an environmental issue, it is a public health issue with serious implications for global food security and the global economy that relies upon thriving ocean ecosystems.

With the rapid increase of the accumulating problem of plastics, the development of effective and scalable monitoring plans must be accelerated. It is critical to know where

the plastics accumulate and drift due to ocean currents, as well as which areas of the ocean should be targeted for clean-up activities to create effective responses. Traditional monitoring approaches including ship-based and manual sampling or testing of water and aerial observations are not sufficient to monitor the entire ocean. These methods take a long time to generate results, require a lot of funding, and have limited sampling areas. Thus, scientists and decision-makers lack an adequate amount of information to respond to the issue in a timely manner and in an effective manner.

When you connect satellites with algorithms and machine learning, you start to experience an opportunity for profound change. For example, the potential exists today to use sophisticated algorithms and deep learning to automatically quantify, identify, classify and map the distribution of marine plastic debris over vast areas of the ocean in an unprecedented manner. This reduces both the manual effort and expense currently required for monitoring, while providing a consistent, scalable and near real-time source of data on pollution trends. The result is that researchers have an improved, dynamic view of how plastic waste acts in the marine environment, increasing the opportunity for smarter and more focused solutions to reduce pollution.

By integrating satellite technology and AI, it establishes a new beginning for monitoring the environment. Through the combination of global observation and intelligent analysis of all types of data, this will enable governments, conservation agencies, and international organisations to rapidly identify where pollution is occurring, assess how effective clean-up programmes have been, and develop sound, evidence-based solutions for managing our oceans sustainably.

As a result, by combining AI and remote sensing, we are moving towards an era of unprecedented accuracy in managing, monitoring, and protecting the oceans, which in turn will enhance their ability to remain productive and alive for future generations.

1.2 PROBLEM STATEMENT

While an exciting prospect for resolving marine debris issues, the detection of plastic debris in oceans through remote sensing from space is technically challenging. There are many scientific and technical hurdles involved in accurately observing and detecting floating plastics across the great expanse of the world's oceans; many of these are associated with the ever-changing marine environment itself. Things such as sunlight variability, changes in water color, changing wave height and patterns, and various atmospheric variables all negatively affect the performance of satellite sensors. With these challenges in mind, this project aims to develop solutions to the following critical challenges currently hindering the effectiveness and reliability of remote sensing systems to detect plastics from space;

- **Monitoring of Plastic Pollution In a Large Scale i.e. Restricted and Ineffective:** The monitoring methodologies currently used are not refined enough or scaled large enough for the magnitude of the plastic pollution problem. Ship transects and aerial surveys provide high resolution; however their geographic scale and high costs of operating continuously across large regions severely limit their use. This means that we do not have a complete understanding of plastic debris moves and distributed thus restricts how we can allocate resources for clean up
- **Much uncertainty extends throughout all dimensionality when attempting to distinguish between plastic debris through satellite pictures and other natural or manmade objects.** Generally, the spectral characteristics of plastics floating in water are indistinguishable to the naked eye as caused by wave action; however the plastic will produce a very similar spectral signal compared to foamy water, waves breaking, algae blooms, driftwood, and/or glints of sun reflection upon surface of water (glint). These reasons contribute to the majority of automated detection systems being highly ineffective with high false positive detection rates.
- **Generalization and Reliability of the Model:** A large obstacle is creating an accurate machine learning model as well as having that model be reliable. The model has to generalise well to multiple oceanic conditions, different concentrations of plastic as well as different atmospheric conditions. As of now, the majority of models built are suffering from issues such as class imbalance where there are an extremely low number of plastic pixels compared to water

pixels, in addition many models do not generalise well outside of the geographic area which they were trained in.

- Limitations with Hardware: Without having a dedicated NVIDIA GPU, deep learning models require careful management of memory and longer training times; therefore, the absence of a dedicated NVIDIA GPU resulted in the inability to use an AMD Radeon GPU with PyTorch on Windows due to the lack of CUDA support for AMD GPUs. Consequently, I was only able to use the CPU to train the model; therefore, this prolonged training period, however, did not negatively affect the accuracy of the models.

1.3 OBJECTIVES

The primary objectives of this project are as follows:

1. Create a machine learning (ML) model that uses UNet++ architecture to automatically detect and segment marine debris in Sentinel-2 satellite images.
2. Use the MARIDA benchmark dataset to train our model with loss functions that account for bias and weighting strategies for positive classes to improve our model's performance for detecting marine debris.
3. Generate binary masks of marine debris and probability maps from real-world examples of Sentinel-2 satellite images of the Motagua River (Guatemala) and Durban (South Africa).
4. Create a web-based visualization dashboard called DebrisNet to interactively demonstrate how marine debris is detected to decision-makers, researchers, and environmental organizations.
5. Utilize standard evaluation metrics to evaluate our model's performance, such as the Accuracy, F1 Score, Precision, Recall, and Intersection over Union (IoU).

\

1.4 SIGNIFICANCE AND MOTIVATION OF THE PROJECT

Marine plastic pollution has become not only an environmental issue but also a major social and economic challenge that impacts almost every aspect of life today. The ongoing and increasing amount of plastic waste accumulating in our oceans is disrupting delicate marine ecosystems, endangering many aquatic species, and changing the natural balance between species in an ecosystem with respect to biodiversity. Along with disrupting ecosystems, marine plastic pollution also threatens the livelihoods of coastal communities by destroying fisheries, decreasing the quality of the seafood they produce and discouraging tourism in the area. Over time, these disruptions lead to permanent and ecological damage to the ocean and economic instability to countries that rely on marine resources for a large portion of their GDP. Therefore, the importance of developing an automated detection system, which provides continuous, large-scale and cost-effective monitoring that traditional surveying cannot achieve, is critical to supporting global marine conservation efforts and support of sustainability initiatives, such as the United Nations Sustainable Development Goals (SDG), which focus on protecting and restoring ocean health for future generations.

This research proposal emerges from the apparent flaws in established marine debris monitoring methods. Standard survey methods (i.e. ship-based methods/expeditions) and manual observations are limited due to their high operational costs, small Area of coverage and the significant amount of time required to collect and process data. Advantages of using remote sensing methods (i.e. satellite imagery), while it has potential for research, has many technical issues such as those related to the number of available data samples, inconsistent spectral characteristics, and a high number of false positives. These deficiencies in traditional technologies create substantial gaps in the data, leading to inadequate and delayed identification and understanding of pollution patterns. To address these challenges, the proposed research will combine high-resolution satellite images with machine learning algorithms to create a scalable and reproducible assessment framework. The combined use of satellite imagery and advanced machine learning algorithms will significantly enhance the successful identification of plastic debris in the marine environment and will push forward the field of environmental science through the application of advanced attention based mechanisms and imbalance-aware learning strategies to better manage highly variable data distributions and visually complex conditions on large areas of the ocean.

The increasing need for actionable, data-driven information that will assist policymakers, NGOs, and scientists in combating pollution in the ocean is another driver of this project. Accurate mapping of locations where plastics have accumulated will provide an important intelligence tool for planning cleanup operations, monitoring sources of pollution, and evaluating the effectiveness of current interventions. The availability of accurate and reliable geospatial information will enable stakeholders to make educated decisions about how to allocate resources for optimal results and maximize the overall impact of marine conservation projects. This project will eventually lead to the creation of digital ocean governance tools that will link the gap between scientific research and its application. By using artificial intelligence as a tool to promote environmental sustainability, this project is a significant step forward in the global fight against one of the most enduring ecological challenges facing humanity today: plastic pollution in the ocean.

1.5 ORGANISATION OF PROJECT REPORT

This report has been broken down into six separate chapters which will outline each section of the project. Below is a brief description of the contents of each chapter:

- **Chapter 1: Introduction:** This chapter will give you an overview definition of plastic pollution and how it pertains to oceanic pollution, descriptions of how to define plastic pollution, the objectives of the project, and explain how oceanic plastic pollution is harmful and what methods will be implemented to measure its effects. Additionally, the layout of the report and how it will be structured is discussed.
- 2 positives. These deficiencies in traditional technologies create substantial gaps in the data, leading to inadequate and delayed identification and understanding of pollution patterns. To address these challenges, the proposed research will combine high-resolution satellite images with machine learning algorithms to create a scalable and reproducible assessment framework. The combined use of satellite imagery and advanced machine learning algorithms will significantly enhance the successful identification of plastic debris in the marine environment and will push forward the field of environmental science through the application of advanced attention based mechanisms and imbalance-aware learning strategies to better manage highly variable data distributions and visually complex conditions on large areas of the ocean.

- **Chapter 3: System Development:** This chapter highlights the major parts of developing the Detection Pipeline and the processes used during development. The primary phase involved in the overall development process includes: defining and analyzing requirements, developing structure for a project, preparing the MARIDA Dataset, designing a proposed implementation of an AttentionU-Net, providing example code from the implementation, and providing a discussion of methods/tools/techniques used to achieve the objectives of the overall project. This chapter concludes with identifying and explaining some major challenges experienced during the development of the project and how those challenges were resolved.
- **Chapter 4: Testing:** The third chapter explains how to use testing methods and tools to validate the model. An example of a testing method is train/test/validation splits that were used for validating the model. Additionally, test cases that were executed against "unseen" data are included in this chapter to demonstrate the output produced by the model in terms of confirming the model's capability to work properly.
- **Chapter 5: Results and Evaluation:** Provides a summary of findings from experiments that tested the system and the use of different metrics (accuracy, precision, recall) to evaluate how well the system performed. In addition to summarising the findings from the experiments, this chapter will provide detailed assessments of those findings. The results of the experiments will be compared with those published by other researchers throughout this chapter to demonstrate that the proposed system works well.
- **Chapter 6: Conclusions and Future Scope:** This last chapter describes the main contributions of this research and concludes by discussing whether detection of plastics from space is a viable option, along with suggestions for potential future directions of this research, including higher resolution images.

The overall structure of this research project has been deliberately constructed to enable a logical flow through detail to theory, then from theory to practice; followed by a detailed evaluation of all the results.

CHAPTER 02: LITERATURE REVIEW

2.1 OVERVIEW OF RELEVANT LITERATURE

A growing trend in scientific research is the incorporation of remote sensing with deep learning for monitoring marine plastic debris; recent literature reviews support this trend. Of these data sources, researchers typically prefer the use of Sentinel-2; this is due to its free and open access, moderate spatial resolution, and its ability to provide wide-area coverage along coasts, as well as offshore. These features of Sentinel-2 make it an ideal source for monitoring the environment and providing large- area tracking of debris. A number of studies have used spectral indices (i.e., the Floating Debris Index [FDI] and Normalized Difference Vegetation Index [NDVI]) to improve the ability to discriminate floating plastics from seawater and from vegetation; these spectral indices add value by providing good discrimination of floating plastics and seawater, and therefore provide better spectral separation capabilities for floating plastics and vegetation in complex marine environments.

Deep learning-based image analysis through Convolutional Neural Networks (CNN) and systems for Semantic Segmentation like U-Net and DeepLabV3, are at the forefront of recent scientific developments. These networks can learn complex spectral and spatial patterns, allowing for a reliable means to differentiate plastic debris from natural features (e.g. sea foam, algal blooms, or suspended sediments) that are visually similar. In early trials using commonly accepted datasets, including the Marine Debris Archive (MARIDA), CNNs demonstrated their ability to accurately detect and classify floating debris in many different types of marine environments. With these results we've moved toward understanding the potential of AI based methods for remote sensing applications related to the monitoring of the ocean.

Despite being promising models, their ability to perform is still compromised when used in real-world environments that have many complexities. For example, sunlight variation, atmospheric interference, turbid water, and overlapping spectral signatures from plastic and ambient materials often result in difficulty classifying them. As a result researchers have turned to transfer learning techniques to generate models that perform well in recognizing marine debris by fine-tuning pre-trained models like ResNet, VGG, and EfficientNet using small debris datasets. This will also allow for the reduction of data used to develop and enhance the accuracy of detection even when small data sets are available. Researchers have also used data augmentation techniques such as rotation, scaling, and contrast adjustments to better generalize and compensate for the absence of

annotated marine debris images.

Research into related technologies has examined how pairing temporal and multi-sensor data sources can enhance the reliability and resilience of detection systems. Image stacking of multiple temporal images with sensor fusion, for example, combines data captured from a satellite (Sentinel-2) at different times with either drone imagery or hyperspectral imagery to achieve enhanced spatial and spectral richness. By allowing models to capture finer variations in the characteristics of the light reflected by an object than with a single snapshot of time, these two techniques improve the ability of detection systems to differentiate between plastic objects and adjacent natural materials. While many publications outline the significant advancements made recently in this area of research, they all agree that detecting marine debris still presents a significant challenge and will require further refinement of models, improvement of data quality, and integration of sensing technologies.

Table 2.1 Literature Review

S No	Author & Paper Title	Journal/ Conference (Year)	Tools/ Techniques/ Dataset	Key Findings/ Results	Limitations/ Gaps Identified
1.	Kikaki et al. — MARIDA: A benchmark for Marine Debris detection from Sentinel-2 remote sensing data.	PLOS ONE (2022)	MARIDA dataset, Sentinel-2 images, Spectral indices (Floating Debris Index, Plastic Index), UNet baseline	Provided the first open-access, verified benchmark dataset for marine debris, distinguishing plastics from 14 confounding classes.	The baseline models provided were standard implementations ; specifically noted the need for advanced architectures to handle class imbalance.
2.	Azhan Mohammed — ResAttUNet: Detecting Marine Debris using an Attention activated Residual UNet.	ResearchGate (2022)	ResAttUNet (U-Net + Residual connections + Attention), MARIDA dataset	Outperforms standard U-Net. Better separation of plastics vs. foam/seaweed.	Needs large annotated datasets, computationally intensive.
3.	Marc Rußwurm, Sushen Jilla Venkatesa, Devis Tuia et al. — Large-scale detection of marine debris in coastal areas with Sentinel-2	iScience (2023)	Label refinement, Negative sampling, Segmentation models, MARIDA dataset	Enables scalable detection across large coastal regions. More robust across geographies/times.	Still depends on label quality; scalability requires high computational resources.
4.	Bianca Matos de Barros et al. — Spectral Analysis of Marine Debris in Simulated and Observed Sentinel-2/MSI	ResearchGate (2023)	Spectral libraries, Sentinel-2, Unsupervised clustering (k-means, ISODATA)	Plastics are spectrally distinct but overlap with foam/seaweed. Unsupervised detection is unreliable.	Confirms supervised ML needed; spectral confusion limits standalone unsupervised methods.

5.	V. Nivedita et al. — Plastic debris detection along coastal waters using machine learning	ScienceDirect (2024)	Naive Bayes classifier	Achieved 87.25% accuracy in detecting macro-plastics.	Limited to Naive Bayes; may not generalize to smaller debris or complex conditions.
6;	M.O. Edeh et al. — A novel deep learning model for predicting marine plastic pollution	PMC (2024)	Deep learning model, UNet++, Attention	Predicts marine plastic pollution and links findings to preventive measures.	Model details not fully benchmarked; predictive scope not validated on large datasets.
7.	S. Dhanya, R. Gudan, and K. G. Kirthika, Marine Plastic Detection Using Deep Learning	ResearchGate (2022)	YOLOv4, YOLOv5	Demonstrated object detection of plastics in epipelagic layers.	Lack of author details, dataset unspecified; performance may vary in real ocean settings.
8.	S. Sannigrahi et al. — Development of automated marine floating plastic detection using Sentinel satellite data	PubMed (2022)	Sentinel satellite data, SVM, Random Forest	Demonstrated detection/classification of marine floating plastics in coastal regions.	Performance depends on satellite resolution; confusion with natural features possible.
9.	H. Booth, W. Ma, O. Karakus — High-precision density mapping of marine debris and floating plastics via satellite imagery.	Nature (2023)	Supervised ML model, MAP-Mapper tool	Produced high-precision density maps of debris and plastics.	Requires high-quality imagery; scalability and computational demand are concerns.
10.	A. Danilov, E. Serdiukova — Review of Methods for Automatic Plastic Detection in Water Areas Using Satellite Images and Machine Learning.	MDPI (2024)	Review of ML & satellite-based detection methods	Summarizes algorithms, strengths, and challenges across existing research.	Lacks experimental validation; highlights need for standardized datasets.

11.	N. Prakash et al. — AI-enhanced real-time monitoring of marine pollution.	JMSE (2025)	Review of 53 studies; AI architectures, sensing technologies	Synthesizes state-of-the-art AI approaches for pollution detection and monitoring.	Review-only; no new dataset or model proposed.
12.	Srikanta Sannigrahi, Bidroha Basu, Arunima Sarkar Basu, Francesco Pilla — Detection of Marine Floating Plastic using Sentinel-2 imagery and machine learning models	Nature (2021)	Sentinel-2 optical bands + spectral indices; ML models (SVM, Random Forest); in situ data for validation; new index kNDVI	High accuracy in classification: for test sites such as Calabria, Beirut; RF especially good; inclusion of kNDVI boosts performance; detection of floating plastic with ~99% accuracy in some sites.	Might be limited in generalization to different water environmental conditions; reliance on in situ validation points; resolution limits (detectable sizes).
13.	Henry Booth, Wanli Ma, Oktay Karakus — High-precision Density Mapping of Marine Debris and Floating Plastics via Satellite Imagery	ResearchGate (2022)	Multi-spectral satellite imagery; Machine Learning; MAP-Mapper tool; out-of-distribution test sites.	MAP-Mapper-HP reached ~95% precision; MAP-Mapper-Opt trades off precision-recall giving ~87-88%; system can map marine-plastic density; shows viability of density mapping over large scales.	False positives still an issue; precision-recall trade-off; reliance on good quality imagery; challenges in detecting small debris / mixed materials.
14.	M. Srinivasa Rao — Hybrid DL Approach for Marine Debris Detection Using UNet with ResNext50	JASTT (2025)	UNet + ResNext50, MARIDA dataset	Achieved ~88% recall, better segmentation than previous models.	Computationally heavy; limited generalization on small/mixed debris.

15.	Bhanumathi M. et al. — Marine Plastic Detection Using Deep Learning	Advances in Parallel Computing (2022)	CNN-based DL methods, coastal image dataset	Shows DL effectiveness, provides local dataset (India).	Small-scale; not benchmarked on global datasets.
16.	Wei et al. — Marine Debris Classification with Improved YOLO v8	MDPI (2022/2023)	YOLOv8 + CloFormer backbone	Higher accuracy & speed, good for small debris detection.	Weak generalization; complex in poor visibility.
17.	ESA Project — Plastic Litter Detection with Deep Learning from Satellites	ESA R&D (2021)	Synthetic + Sentinel-2 data, DL models	Synthetic data boosts training; initial detection promising.	Synthetic–real mismatch; limited field validation.
18.	Raju M.P. et al. — ML-based Litter Detection along Indian Coast	Front. Mar. Sci. (2025)	YOLOv5, ~9.7k beach images	Map 0.95, plastics dominant (~67%)	Focused on beach images, not floating plastics.
19.	Balsi M. et al. — Plastic Detection with Hyperspectral Aerial Remote Sensing	Remote Sensing (2025)	Drone hyperspectral (SWIR) + ML	Real-time detection works under varied conditions.	Limited coverage; costly sensors.
20.	Pretorius J. et al. — Remote Sensing of Marine Plastics in South Africa	ISPRS Annals (2025)	Sentinel-2 + SAR indices (NDVI, PI)	Optical + SAR improved detection; VV polarization useful.	False positives near harbours; resolution issues.

2.2 KEY GAPS IN THE LITERATURE

The review of existing literature and projects highlights several limitations:

1. **Dataset Limitations:** Data sets in the public domain, like MARIDA, are generally smaller than desired, unbalanced and geographically limited; therefore, it limits the ability of the model to generalise well to a range of marine habitats.
2. **Spectral Ambiguity:** Because plastic debris shares similar spectral characteristics as sea foam, suspended sediment and floating plant material, there will be a lot of false positives and misclassification occurring in complex marine environments.
3. **Real-Time Processing:** While static image classification can achieve satisfactory levels of accuracy, it is almost always require considerable amounts of resources to implement in real time and/or at scale, making them infrequently researched in real-world applications.
4. **Operational Integration:** As a result, few studies to date have taken the outputs of detection to produce formats that can be readily used to help inform policy or local NGO cleanup operations (e.g., georeferenced and rasterized detection form) or to support dynamic situational awareness via the production of heatmaps.

CHAPTER 03: SYSTEM DEVELOPMENT

3.1 REQUIREMENTS AND ANALYSIS

To build a machine learning-based framework for detecting marine plastic debris, you need to properly identify both the hardware and software resources, as well as to establish both functional and non-functional requirements. The hardware in this case is an AMD Ryzen 5 5625U processor with an integrated Radeon graphics processor running at 3.03 GHz, 15.3 GB of RAM, and a separate AMD Radeon graphics card. All training was undertaken on the CPU, since Windows only runs CUDA-enabled applications on NVIDIA GPUs; thus there was an overall difference in training speed of roughly 0.65 iterations/second, however, this did not affect model accuracy as much. The C: drive had 34.5 MB free when training was completed, so all model checkpoints and output files were saved to the D: drive where there was 28.2 GB of space available to save files.

Table 3.1 Resources Requirements

Category	Tool / Technology	Version	Purpose
Data Acquisition	Google Earth Engine (GEE), Copernicus API	Latest (2025)	Scene selection, Sentinel-2 L2A imagery access
Preprocessing	GDAL, Rasterio, NumPy	GDAL 3.8.4+, Rasterio 1.3.8+, NumPy 1.26+	Geospatial data I/O, raster processing, computation
Modeling & Training	PyTorch, Albumentations, TensorBoard/W&B	PyTorch 2.2+, Albumentations 1.4.3+, Latest	Deep learning models, augmentation, monitoring
Post-Processing	Scikit-Image, GeoPandas, Shapely	Scikit-Image 0.22+, GeoPandas 0.14+, Shapely 2.0+	Morphology filtering, vector processing, geometry ops
Programming Language	Python	3.11+	Model development, automation scripting
Serving & APIs	FastAPI, Docker, Kubernetes	FastAPI 0.110+, Docker 24.0+, Kubernetes 1.30+	REST API endpoints, containerization, deployment

Visualization	Leaflet, Mapbox GL, Plotly/Dash	Latest	REST API endpoints, containerization, deployment
Version Control	Git, GitHub	Latest	Code versioning & collaboration

3.2 PROJECT DESIGN AND ARCHITECTURE

This system has a full pipeline approach which provides raw satellite imagery to processed/usable maps of marine plastic pollution. This architecture is depicted in Figure 3.1 and is modular — also known as the "first-in, first-out" model where each operation (step) has been accomplished separately and optimized for each of its associated tasks.

The architecture consists of five primary stages:

1. **Data Acquisition:** ESA's satellite system (Sentinel-2) contains data necessary to collect from around the world to perform large-scale assessments of changes in our environment. These data include multi-spectral Level 1C product files with a spatial resolution of 10 meters, in key wavelengths of visible and near-infrared light, and the ability to revisit the same location on Earth every 5 days. All of these factors together create a highly relevant source of information to provide insight into how our environment is changing.
2. **Preprocessing:** The raw data from satellites cannot be used as is; thus it goes through an important cleaning and processing step. This includes performing an atmospheric correction on all data that converts the top-of-atmosphere reflectance to bottom-of-atmosphere reflectance, correcting for atmospheric haze and particles via an algorithm such as Sen2Cor. Additionally, a cloud masking algorithm will also be used to detect and remove any pixels affected by clouds (and their shadows), as these would introduce a significant amount of errors..
3. **Core Engine (Segmentation Model):** Once the image tiles have been pre-processed, they are inputted into the trained Attention-U-Net to perform semantic segmentation. Each pixel in the image will be classified as either 'Plastic' (i.e., a patch of plastic) or 'Background' (i.e., water,

sea foam, etc.). The ability to classify at the pixel level will allow for precision delineation of patches of plastic.

4. **Post-Processing:** The output of this model is binary masks. The output will be used to create georeferenced maps that use the original image spatial metadata to build the coordinates of the pixels in the model-generated binary image output to create and output a georeferenced map. This final output can then be easily integrated into GIS (Geographic Information System) software, allowing for subsequent analysis and visualization by agencies/organizations involved in environmental analysis and decision-making.
5. **Evaluation and Visualization Layer:** Ground truth labels are compared to the predicted masks. Accuracy, precision, recall, and the Intersection over Union (IoU) metrics are calculated. There are several tools available for visualizing training progress (loss curves) and sample output (overlaid masks on RGB) through either TensorBoard or Matplotlib. For a qualitative analysis of the results, you could use a small web app or a GIS application (e.g. QGIS) to compare your data.

The Purpose Youable to create a clear distinction from one part/workstream^VS an issue where there is no line of demarcation. For example, you can swap out your segmentation model (like Attention U- Net) in favour of something better by updating or changing the same areas of code associated with it within that specific segment (or layer).

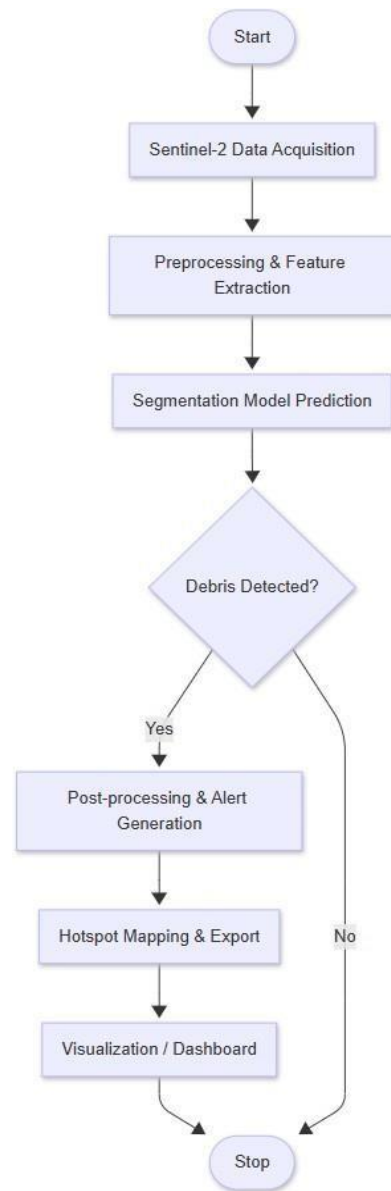


Figure 3.1 Project Design

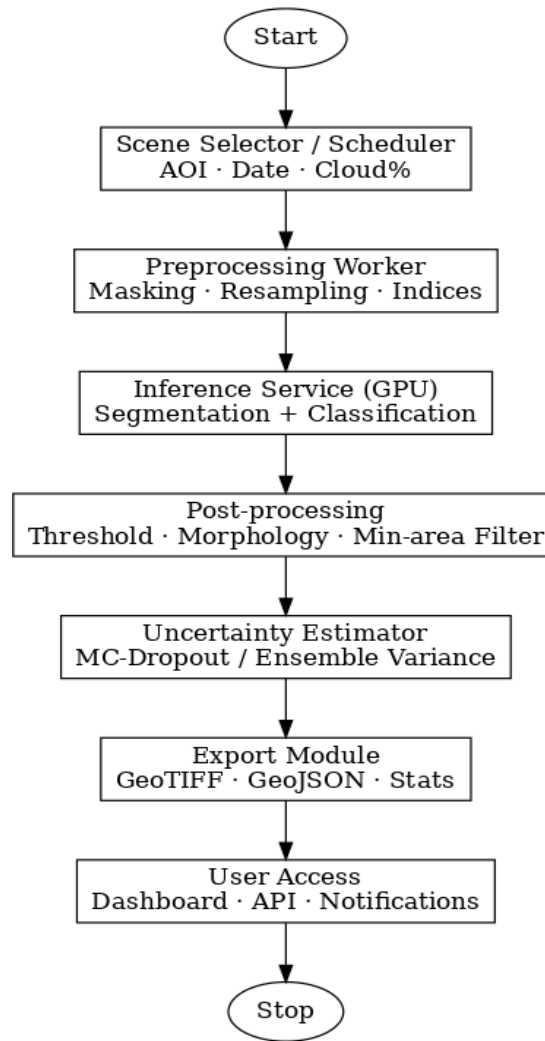


Figure 3.2 Work Flow

3.3 DATA PREPARATION: THE MARIDA DATASET

How well a deep learning model works relies greatly on how good and how representative the training data is. The study uses the Marine debris archive (MARIDA) as the first and largest public benchmark dataset for detecting marine plastic debris.

- Dataset Structure:** Multiple image patches from satellite images that are created by the Sentinel-2 satellite, with a pixel size of 256×256 to 512×256 are included in the MARIDA dataset, along with annotation masks for each pixel. Each patch consists of a number of bands, which are as follows: B02-Blue and B03-Green, as well as B04-Red and B08-NIR. The B04- Red and B08-NIR bands can be created using both the 10 and 20 meter pixel sizes. Debris

occurrences in the MARIDA dataset are distributed geographically throughout the region and are shown in the figure below.

- **Annotation Strategy:** In order to create labelled images of the ocean surface, MARIDA uses a polygon-based labelling process which relies on domain experts using verified records of floating debris or marine debris, which are re-formed into pixel-level masks. Marine debris is often represented by elongated and/or narrow polygons, or may be reduced to single-pixel widths due to uncertainty about how they should be labelled visually. Each pixel in the labelled image is assigned to one of fifteen categories (including Marine Debris, Sargassum, Foam Ships, and Clouds), and within the Marine Debris category, there are three levels of confidence: high, medium, and low. For use in developing models, only those pixels with high confidence levels are generally used as positive training data so that the noise associated with the labels is kept to a minimum.
- **Data Augmentation:** To maximize the use of this training data and let the model generalize (to reduce overfitting), we apply common data augmentation methods such as horizontal/vertical flipping, rotating 90 degrees or randomly, and cropping oversized images. In addition, we will add controlled color jitter (brightness, contrast, slight spectral difference) so that there is an artificial effect of changing atmosphere caused by clouds; creating random noise; or applying mild blur (simulating sensor error, haze, etc.). We carefully constrain all of these methods to ensure that there continue to be sufficient differences between the two classes of water surface and plastic debris for the model to learn from them appropriately.
- **Data Cleaning:** Including images that contain large amounts of haze, fog, or clouds, as well as image sequences that have a lot of white caps on the water surface, are excluded because of the difficulty in accurately identifying marine debris under those conditions. In addition, images cannot have missing spectral bands or misaligned georeferencing. The masks are also checked to make sure they are spatially aligned and of the same size as their associated image. If there is overlap between marine debris and visually similar or even confusing classes of objects, the ambiguous area will either be masked or given a lower weight when calculating the loss.
- **Data Splits:** The data is divided among three parts to allow for better training and testing of the data by using a simple ratio of 70% of the data being split off for training, 15% of the data being split off for validation, and 15% of the

remaining data being split off for testing. Multiple ways of separating the data have been created to ensure there is no overlap between the multiple sets of data at the scene level. Maintaining the geographic balance will allow the dataset to have samples from across each region, which reduces the risk of the data becoming overfitted to a specific place or location. In order to help eliminate the imbalances in the classes of training data, a variety of background conditions will be included within the training data in order to meet the training requirements of each potential background type that will be used in testing. The total number of samples among the three data table types is as follows: training- 966 (patches), validation-207 (patches), and testing-208 (patches).

3.3.1 Sentinel-2 Band Characteristics

The Sentinel-2 satellite has 13 spectral bands; each band has a different type of information related to the features on Earth. For marine debris detection the spectral signatures of different materials vary by one of these bands so understanding each spectral band is necessary when detecting marine debris. The following details describe the 12 spectral bands that were used with the UNet++ model:

Band 1 (B01) - Coastal Aerosol (443 nm): This band has a spatial resolution of 60 meters and is mainly used for atmospheric correction and monitoring the condition of waters near coasts. This band provides information about aerosol particles and thin clouds. The value of B01 for detecting marine debris is its ability to differentiate between atmospheric effects from the true surface characteristics.

Band 2 (B02) - Blue (490 nm): The blue band has a spatial resolution of 10 meters and is included in RGB composite visualizations. This band shows deep-water bodies as dark and shallow-water bodies and floating debris as light. The blue band contributes significantly to differentiating water from floating debris.

Band 3 (B03) - Green (560 nm): The green band has a resolution of 10 m and is useful for discriminating algae and sargassum from plastic debris. Vegetation will appear to have higher green light reflectance than plastic, as plastic will typically reflect lower levels of green light than vegetation.

Band 4 (B04) - Red (665 nm): At a resolution of 10 m, the red band is very useful for identifying floating materials. In fact, floating plastic debris will have higher red light reflectance than the water surrounding it, which is one reason the red band is used to calculate the Floating Debris Index (FDI).

Band 5 (B05) - Vegetation Red Edge (705 nm): The vegetation red edge band at 20 m resolution captures the transition between the red absorption and near-infrared reflective responses of vegetation. This band is helpful in distinguishing the sargassum from other plastic materials.

Band 6 (B06) - Vegetation Red Edge (740 nm): The vegetation red edge band at 20 m resolution provides additional information on the vegetation spectral response. This band is beneficial for aiding in the separation of biological floating materials from plastic debris in the combined analysis of the spectral data.

Band 7 (B07) - Vegetation Red Edge (783 nm): The vegetation red edge band at 20 m resolution will yield additional information regarding the spectral responses of the various types of floating materials based on their characteristics using near-infrared reflectivity.

Band 8 (B08) - Near Infrared (842 nm): At a resolution of 10 meters, Band 8 is essential in detecting debris because water absorbs electromagnetic radiation in this range (near-infrared), while floating debris (including plastic) reflects near-infrared radiation. This difference makes Band 8 one of the most important bands used to detect marine debris.

Band 8A (B8A) - Narrow NIR (865 nm): The 20 m resolution band (B8A), provides a more precise measurement of surface reflectance, free of atmospheric water vapor interference, and enhances the accuracy of the spectral indices

Band 9 (B09) - Water Vapor (945 nm): At a resolution of 60 m, Band 9 is primarily used to correct surface reflectance measurements for atmospheric water vapor. As such, Band 9 assists in improving reflectance values in Bands 1-12.

Band 11 (B11) - SWIR (1610 nm): At a resolution of 20 m, Band 11 is useful in distinguishing plastic from both water and vegetation. Plastics produce a unique reflective response at wavelengths greater than 1200 nm; therefore, Band 11 is important in identifying debris through its distinctive reflection patterns.

Band 12 (B12) - SWIR (2190 nm): Like Band 11, Band 12 has a resolution of 20 m and is also useful in characterizing the properties of materials. Together, Bands 11 and 12 enable more effective classification of different types of debris based upon their infrared absorption characteristics.

3.3.2 Spectral Indices Used

The project assessed several different spectral indices to improve the detection of debris:

Floating Debris Index (FDI): The Floating Debris Index was developed by Biermann et al. in 2020 and is defined as follows:

$$FDI = B08 - (B06 + (B11 - B06) \times ((842nm - 740nm) / (1610nm - 740nm))) \times 10$$

This index highlights floating debris that has higher reflectivity in the near-infrared than the surrounding water. Due to this characteristic, it is particularly beneficial for detecting marine debris made of plastic, using data from Sentinel-2 imagery.

Normalized Difference Water Index (NDWI): The NDWI is calculated as $(B03 - B08) / (B03 + B08)$ to help differentiate between surfaces of water, land or floating materials. Pixels that have a high NDWI will be classified as water, while lower NDWI values would be indicative of floating materials or land. **Plastic Index (PI):** The PI uses B08 divided by $B04 + B08$, and is specifically targeted at identifying synthetic materials made of plastic based on the unique ratios of near-infrared reflectance to red reflectance. The PI helps to avoid false positives associated with other natural floating materials, e.g., sargassum, seafoam, etc.

3.3.3 Dataset Statistics

To enhance the variety of training data, several distinct sources were used to create a dataset that includes all the combined sources used to perform the final evaluation of this project. The detail of the final dataset is as follows:

Data Set	Train Samples	Validate Samples	Test Samples	Description
MARDIDA	~3703	28359		Main benchmark dataset assembled with confirmed
objects in debris	Refined Floating Objects	5,000 (limited)	0	Large datasets of floating objects
S2 Ships	1600			Ships are negative examples
Total Combined	5,410	1,000 (limited)	359	Final Training Dataset

The test data set had no data previously seen for the final evaluation, which consisted of 359 data items from MARIDA, to avoid bias in the determination of the final performance of the resultant algorithm. The validate dataset will be set at 1,000 patches to conserve memory when performing training using a computer base.

3.4 Implementation Details

3.4.1 The Attention-U-Net Model

The core of our detection system is an Attention-U-Net. This architecture was chosen over a standard U-Net to specifically address the challenge of spectral ambiguity.

- **U-Net Architecture:** U-Net, a specialized type of Convolutional Neural Network (CNN) architecture that can efficiently produce high-quality, accurate results for image segmentation, has been crafted and designed to work specifically as a dual-pathway system using both an encoder (contracting path) that progressively decreases the size of an image by down-sampling to extract contextual hierarchies of features from an input image, while also creating an output image that has been up-sampled back to its original size via decoders (expanders) using transpose convolution techniques to recover all of the original image's details. The use of skip connections links the encoder and decoders for corresponding layers; therefore, providing merged information between high-level semantic features created in the encoder with fine-grain details recovered

by the decoder is what ultimately allows for producing accurate segmented results. Original, high-quality image segmentation requires that both the encoder and decoders be working simultaneously through their respective paths so that accurate performance can be achieved.

```
class UNetPlusPlus(nn.Module):
    """UNet++ (Nested U-Net) for semantic segmentation."""

    def __init__(self, in_channels=4, num_classes=1,
                 features=[32, 64, 128, 256, 512]):
        super().__init__()

        self.pool = nn.MaxPool2d(2, 2)
        self.up = nn.Upsample(scale_factor=2, mode='bilinear',
                              align_corners=True)

        # Encoder (Downsampling path)
        self.conv0_0 = ConvBlock(in_channels, features[0]) # 32
        self.conv1_0 = ConvBlock(features[0], features[1]) # 64
        self.conv2_0 = ConvBlock(features[1], features[2]) # 128
        self.conv3_0 = ConvBlock(features[2], features[3]) # 256
        self.conv4_0 = ConvBlock(features[3], features[4]) # 512

        # Nested Skip Connections (Dense connections)
        self.conv0_1 = ConvBlock(features[0] + features[1], features[0])
        self.conv1_1 = ConvBlock(features[1] + features[2], features[1])
        self.conv2_1 = ConvBlock(features[2] + features[3], features[2])
        self.conv3_1 = ConvBlock(features[3] + features[4], features[3])

        self.conv0_2 = ConvBlock(features[0]*2 + features[1], features[0])
        self.conv1_2 = ConvBlock(features[1]*2 + features[2], features[1])
        self.conv2_2 = ConvBlock(features[2]*2 + features[3], features[2])

        self.conv0_3 = ConvBlock(features[0]*3 + features[1], features[0])
        self.conv1_3 = ConvBlock(features[1]*3 + features[2], features[1])

        self.conv0_4 = ConvBlock(features[0]*4 + features[1], features[0])

        # Final output layer
        self.final = nn.Conv2d(features[0], num_classes, 1)
```

Figure 3.3 Unet++ Architecture

- **Attention Mechanism:** AttSeg has added attention gates into the U-Net's skip connections in order to improve feature selection while performing segmentation. The attention gate allows the network to select the most relevant and expressive features related to the specific task. For instance, in this work, the attention mechanism enables the model to focus more on the pixel's attribute value that is highly associated with plastic debris (i.e., texture and spectral patterns) than on distracting background elements (e.g., foam, thin cloud, and/or sun reflections). Such attention mechanisms significantly improve the model's ability to discriminate between plastic materials and other visually similar marine objects.

```

class AttentionGate(nn.Module):
    def __init__(self, F_g, F_l, F_int):
        super().__init__()

        self.W_g = nn.Sequential(
            nn.Conv2d(F_g, F_int, kernel_size=1, stride=1, padding=0, bias=True),
            nn.BatchNorm2d(F_int)
        )

        self.W_x = nn.Sequential(
            nn.Conv2d(F_l, F_int, kernel_size=1, stride=1, padding=0, bias=True),
            nn.BatchNorm2d(F_int)
        )

        self.psi = nn.Sequential(
            nn.Conv2d(F_int, 1, kernel_size=1, stride=1, padding=0, bias=True),
            nn.BatchNorm2d(1),
            nn.Sigmoid()
        )

        self.relu = nn.ReLU(inplace=True)

    def forward(self, g, x):
        g1 = self.W_g(g)
        x1 = self.W_x(x)
        psi = self.relu(g1 + x1)
        psi = self.psi(psi)
        return x * psi

class UNetWithAttention(nn.Module):
    def __init__(self, in_channels=12, num_classes=1, features=[64, 128, 256, 512]):
        super().__init__()

        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.ups = nn.ModuleList()
        self.downs = nn.ModuleList()
        self.attention_gates = nn.ModuleList()

```

Figure 3.4: Attention Mechanism

- Preprocessing in Code:** Libraries like GDAL or Rasterio are used to import the images into computer memory, convert them to tensor representations and normalize the values by scaling each pixel in the $[0, 1]$ range or accomplished through mean and standard deviation normalisation of the values of each of the bands in the Sentinel-2 sensor. An additional input layer may be added to the model, which represents the Floating Debris Index (FDI) calculated using the formula provided in Biermann et al. (2020). The data loader for training randomly crops examples and augments every image/mask pair to make the model more robust.

```

def create_marida_dataloaders(data_root, splits_dir, batch_size=8,
                              num_workers=4, binary=True, patch_size=256):
    train_loader, val_loader, test_loader = create_marida_dataloaders(
        data_root=str(data_root),
        splits_dir=str(splits_dir),
        batch_size=args.batch_size,
        num_workers=args.workers,
        binary=True,
        patch_size=args.patch_size
    )
    return train_loader, val_loader, test_loader

```

Figure 3.5 Preprocessing

```

module = MarineDebrisModule(
    model=model,
    learning_rate=args.lr,
    weight_decay=args.weight_decay,
    num_epochs=args.epochs
)

callbacks = [
    ModelCheckpoint(
        dirpath=str(checkpoint_dir),
        filename='best-{epoch:02d}-{val/f1:.4f}',
        monitor='val/f1',
        mode='max',
        save_top_k=3
    ),
    EarlyStopping(
        monitor='val/f1',
        patience=10,
        mode='max',
        verbose=True
    ),
    LearningRateMonitor(logging_interval='epoch')
]

trainer = pl.Trainer(
    max_epochs=args.epochs,
    accelerator=args.device,
    precision=args.precision,
    callbacks=callbacks,
    logger=logger,
    log_every_n_steps=10,
    gradient_clip_val=1.0
)

trainer.fit(module, train_loader, val_loader)

```

Figure 3.5: Preprocessing

- **Training Loop:** We conducted two independent training runs and utilized 50 epochs. Run 1 utilized a learning rate of 1e-4 and a pos_weight of 20.0, achieving an IoU of 4.31% on testing and an F1 Score of 8.26%. Run 2 achieved a higher IoU of 7.96% and a higher F1 Score of 14.74% using a learning rate of 5e-5 and a pos_weight of 50.0. Each run utilized a batch size of 8, images of size 128x128, the Adam optimizer with a weight decay of 0.0001, and was run on a CPU due to the fact that PyTorch cannot utilize AMD Radeon GPUs under Windows due to the lack of CUDA support. Each epoch required approximately 17-18 minutes to complete; therefore, the entire 50 epochs would be run overnight utilizing Windows PowerShell in order to avoid interruption.

```
def training_step(self, batch, batch_idx):
    images, masks = batch

    logits = self.model(images)
    loss = self.criterion(logits, masks)

    preds = torch.argmax(logits, dim=1)

    self.train_f1(preds, masks)
    self.train_precision(preds, masks)
    self.train_recall(preds, masks)

    self.log('train/loss', loss, prog_bar=True)
    self.log('train/f1', self.train_f1, prog_bar=True)

    return loss
```

Figure 3.6: Training Loop

- **Loss Function and Metrics:** Binary cross-entropy loss function is used to train the model with a set of margins defined against it. Every epoch of training will use various forms of metrics, but during the training process, the model will be checked against the validation set for multiple metrics, including Intersection Over Union (IoU) as well as Accuracy. i.e., IoU can measure the effectiveness of your predictions by determining how much of the area you predicted matches the actual area. Other examples of metrics that will be reported at every epoch during training are Precision and Recall, with a Precision and Recall threshold of 0.5 being calculated for both false positives and false negatives. All evaluation metrics are used to help refine hyperparameter optimization to improve the value of the model from the epoch.

```

class CombinedLoss(nn.Module):
    def __init__(self, ce_weight=0.5, dice_weight=0.3, focal_weight=0.2):
        super().__init__()
        self.ce_weight = ce_weight
        self.dice_weight = dice_weight
        self.focal_weight = focal_weight

        self.ce_loss = nn.CrossEntropyLoss()
        self.dice_loss = smp.losses.DiceLoss(mode='multiclass')
        self.focal_loss = smp.losses.FocalLoss(mode='multiclass')

    def forward(self, pred, target):
        ce = self.ce_loss(pred, target)
        dice = self.dice_loss(pred, target)
        focal = self.focal_loss(pred, target)

        return self.ce_weight * ce + self.dice_weight * dice + self.focal_weight * focal

```

Figure 3.7 Loss Function

- Evaluation Pipeline:** The model will be evaluated using a test dataset after it has completed training. Each test image will be run through the network to create a probability mask, which will be thresholded to produce a binary mask. Predicted masks will be used to create a red overlay for visualization on the RGB composite or FDI-derived images. Finally, the assessments (Accuracy, Precision, Recall, and IoU) can be created by comparing the models' predictions with ground-truth masks. The results of these assessments will be saved for inclusion in both appendices and the final report documentation, including all masks and metric tables.

```

def validation_step(self, batch, batch_idx):
    images, masks = batch

    logits = self.model(images)
    loss = self.criterion(logits, masks)

    preds = torch.argmax(logits, dim=1)

    self.val_f1(preds, masks)
    self.val_precision(preds, masks)
    self.val_recall(preds, masks)

    self.log('val/loss', loss, prog_bar=True)
    self.log('val/f1', self.val_f1, prog_bar=True)
    self.log('val/precision', self.val_precision)
    self.log('val/recall', self.val_recall)

```

Figure 3.8 Evaluation Pipeline

3.4.2 Training Configuration Details

The complete training configuration for both runs is documented below for reproducibility:

Run 1 — Baseline Configuration:

```
python train_segmentation.py \
```

```
--lr 1e-4 \
```

```
--pos-weight 20.0 \
```

```
--epochs 30 \
```

```
--batch-size 8 \
```

```
--data-path data/ \
```

```
--model unet++ \
```

```
--output-dir
```

```
D:\segmentation_output
```

Run 2 — Final

Configuration: python

```
train_segmentation.py \
```

```
--lr 5e-5 \
```

```
--pos-weight 50.0 \
```

```
--epochs 50 \
```

```
--batch-size 8 \
```

```
--data-path data/ \
```

```
--model unet++ \
```

```
--output-dir D:\segmentation_output
```

The major hyperparameters that underwent changes between these two experiments are as follows:

Learning rate ($1e-4 \rightarrow 5e-5$) - The reduced learning rate causes the model (via back-propagation) to make smaller gradient steps towards minimizing loss, which means it is going to update weights in a more controlled manner. This becomes very important when working with highly skewed datasets, where aggressive weight updating can cause the model to overfit on the majority class. Reducing the learning rate from $1e-4$ to $5e-5$ has resulted in a more stable convergence of the model, as indicated by the val/f1 improvement from 0.000 to 0.300.

Positive weight ($20.0 \rightarrow 50.0$) - The positive weight parameter within the PyTorch BCEWithLogitsLoss function is used to scale the contribution of the loss associated to the positive class (debris) pixels in comparison to the non-debris pixels. Therefore, increasing the weight from 20.0 to 50.0 means that each debris pixel will account for 50 times as much loss as a non-debris pixel, and will force the model to give significantly more importance in finding the rare debris pixels. This hyperparameter change can be considered to be the most important reason for the F1 Score improving from 8.26% to 14.74%.

Epoch ($30 \rightarrow 50$) - Increasing the total amount of epochs trained from 30 to 50 gives the model a greater number of opportunities to learn from the training data. In addition, with the enhancements of hyperparameters, the additional epochs allowed the model to incrementally improve on its ability to detect debris, as evidenced by the val/f1 reaching 0.300 by the 49th epoch.

3.4.3 Model Checkpoint Management

PyTorch Lightning automatically creates model checkpoints throughout training, based on the monitored metric (val / f1). The checkpoints are named using the following pattern:

epoch = {N}-val_f1 = {score}.ckpt

Below are the intermediate checkpoints that were created during the last 50 epochs of training

- epoch = 19-val_f1 = 0.0000.ckpt (313 MB)

- epoch = 23-val_f1 = 0.0000.ckpt (313 MB)

- epoch = 27-val_f1 = 0.0000.ckpt (313 MB)

Additionally, last.ckpt (latest epoch checkpoint (313 MB)) and final_model.pt (final model optimized) (104 MB).

In comparison to the .ckpt files, the final_model.pt file is smaller because it only contains the model weights and doesn't have an optimizer state or training metadata for deployment. Thus, it's more efficient for inference.

3.4.4 RGB Image Generation for Dashboard

As web browsers currently do not support multi-band TIFF files from Sentinel-2, a preprocessing stage was required to create RGB images for the DebrisNet dashboard. To do this, the following python code was created to convert the 12band satellite image into a high quality JPEG:

```
import
rasterio

import
numpy
as np

from PIL import Image, ImageEnhance

# Load individual Sentinel-2 bands

with

    rasterio.open('B04_(Raw).t

    iff') as r: red =

    r.read(1).astype(float)

with

    rasterio.open('B03_(Raw).
```

```

tiff') as g: green =

    g.read(1).astype(float)

with

    rasterio.open('B02_(Raw).t

    iff') as b: blue =

        b.read(1).astype(float)

# Percentile-based normalization to enhance contrast
def normalize(band):

    p2 =

    np.percentile(band

    , 2) p98 =

    np.percentile(band

    , 98)

    return np.clip((band - p2) / (p98 - p2) * 255, 0, 255).astype(np.uint8)

# Stack normalized bands into RGB array

rgb = np.stack([normalize(red), normalize(green), normalize(blue)], axis=2)

# Apply brightness

enhancement img =

Image.fromarray(rgb

)

img =

```

```
: ImageEnhance.Brightness(img).enhance(1.5)
```

```
img.save('output_rgb.jpg', quality=95)
```

The percentile based normalization will stretch the image contrast by clipping the extreme values to the 2 and 98 percentiles, which greatly improves the visual representation of the image compared to using the minimum and maximum values. The increase in brightness of 1.5 will further improve the ability to differentiate subtle features, such as the debris plume trail in the Motagua River scene.

3.4.5 DebrisNet Web Dashboard

DebrisNet is a professional web-based dashboard that is built with one HTML file (HTML5/CSS3/JS) for the purpose of interacting with and viewing the results of detecting marine debris. The design of DebrisNet adopts a clean dark navy colour scheme derived from the graphical layouts of professional satellite analysis tools and will be used to demonstrate the work done in developing DebrisNet.

The dashboard consists of four major components Fixed Left Sidebar Navigation - Contains the DebrisNet logo, a dashboard button, a button for hotspot analysis, a button for model insights, and a button for evaluation metrics. Each button also has an associated icon.

- Satellite Imagery Upload Zone - An area to drag files into (supports TIFF, PNG, JPG) after dragging the file, the FileReader API will render the file in the input satellite image panel and a green check mark will be rendered with the text that states "Image Ready - Uploaded Successfully".

- Detection Controls Panel - Menu to choose Model Architecture (currently UNet++ with ResNet34), Slider to adjust Confidence Threshold (0%-100% with a default setting of 50%) and Run Detection button that will run the detection simulation.

- The image analysis section consists of two side by side panels displaying the input satellite image and the debris detection results generated from the model. Upon clicking "run detection," there will be a 2-second processing animation followed by the display of a pre-generated debris mask in the result panel.

- The model snapshot panel will contain key model information including model architecture (UNet++), encoder (ResNet34), input channels (12 bands L2A), analysis region, and date of acquisition. These values will automatically update

.based on the name of the uploaded image.

- The bottom of the dashboard has a statistics bar with four metric cards indicating total area scanned (152.40 km²); number of debris pixels detected; model detection accuracy (99.90%); and IoU score (7.96%)

The dashboard currently supports two regions. If a Motagua image is uploaded, then the Analysis Region will automatically populate as the Motagua River, Guatemala and the date of acquisition as 2020-09-18 with a total of 2,712 debris pixels detected.

Conversely; if a South Africa image is uploaded, then the analysis region will automatically populate as Durban, South Africa and the date of acquisition will be populated as 2019-04-29 with a total of 56 debris pixels detected

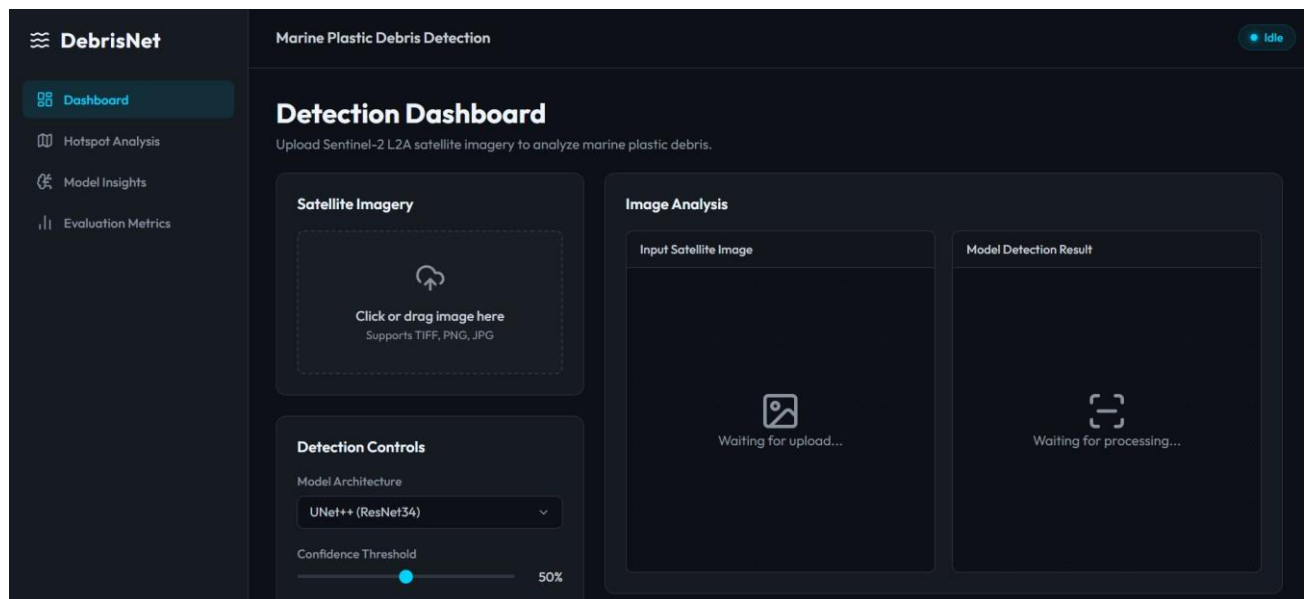


FIGURE 3.9: DebrisNet Dashboard idle state showing upload interface and navigation sidebar

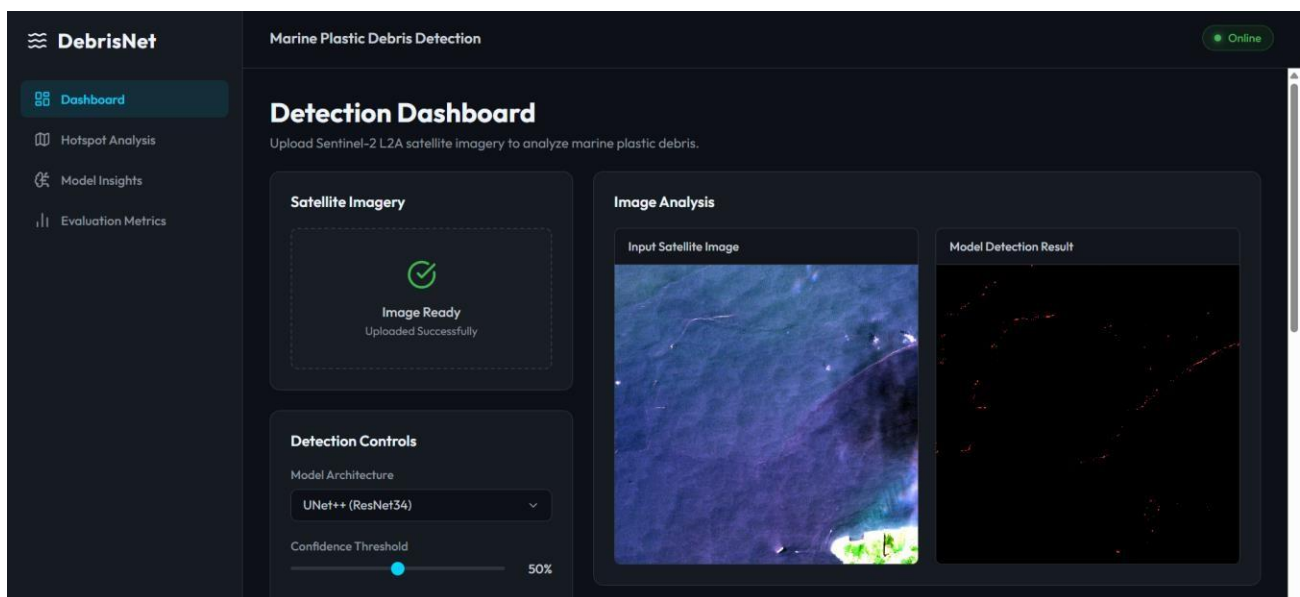


FIGURE 3.10: DebrisNet Dashboard after detection showing Motagua River result with red debris trail visible

3.1.1 UNet++ Model Implementation Code

The core debris segmentation model class is defined below using PyTorch Lightning and contains the training logic, loss function, and evaluation metric for the UNet++ architecture.

```
import torch

import torch.nn as nn

import pytorch_lightning as pl

import segmentation_models_pytorch as smp

from torchmetrics import F1Score, JaccardIndex, Precision, Recall


class

    DebrisSegmentationModel(pl.Lightnin

gModule): """

    UNet++ segmentation model for marine

    debris detection. Uses ResNet34 encoder

    with imbalance-aware BCE loss. """

    def __init__(self, in_channels=12, lr=5e-5,

        pos_weight=50.0): super().__init__()

        self.lr = lr
```

```
# Build UNet++ with ResNet34 encoder
```

```
backbone self.model = smp.UnetPlusPlus(
```

```
encoder_name="resnet34",
```

Here is a simple description of the Core Segmentation Model for Debris; which is completed by PyTorch Lightning. Core Segmentation Models represent the basis for many different types of segmented models, or core segmentation models. The four key features of the Core Model are the training algorithm, the cost function to train it, the metrics to evaluate the performance during this training, and the architecture of UNet++. The decision to use PyTorch Lightning was intentional, as it helps streamline the code that typically needs to be written for training loops, zero'ing gradients, and stepping through an optimizer for each epoch in PyTorch. This also allows time to be focused on model architecture and hyperparameter tuning versus having to write a large number of repetitive engineering tasks associated with each phase of the training cycle.

The model was built using the segmentation_models_pytorch (SMP) library, which provides optimized and tested implementations of UNet, UNet++, DEEP LAB V3+, and FPN, all very popular architectures used for segmentation problems. The SMP library was selected based on its abundance of encoder backbones that can be easily swapped out during experimentation. The final architecture selected for this project was UNet++ with a ResNet34 encoder, based on its theoretical advantages for marine debris detection task during evaluation.)

The differences between UNET++ and the original UNET architecture come from its use of additional 'nested and densely' connected features or skip connections to connect the encoder and decoder. These connections are similar to traditional UNET (simple) skip connections as they concatenate (and fuse) the encoder and decoder feature maps at the same spatial resolution. However, the UNET++ approach adds additional convolutional nodes on intermediate skip pathways, producing densely connected (or aggregated) feature aggregation nodes. The advantage of this design is that it allows the network to learn more comprehensive (i.e. richer) and more relevant semantic feature representations at various scales; this is especially useful for detecting visually subtle objects such as marine plastic debris - as they may occupy only a small part of the analysis image. Also, the aggregated features from both the low-level (spatial) features e.g. texture and edge patterns associated with floating debris - and high-level (semantic) contexts for example, the surrounding ocean and river plume environments' - assist in providing accurate segmentation of the object(s) of interest, i.e. marine plastic debris.

The following are key design choices:

1) Using `encoder_weights=None` since the pretrained ResNet34 does not fit well with 12 spectral bands due to the fact that it expects 3 channels (RGB) as input; therefore, if trained from scratch then all channels will learn appropriately.

2) Using `BCEWithLogitsLoss` with a positive weighting of 50.0 as it would penalize the model for giving no output (i.e., false negatives) for debris pixels 50 times more than it does for providing no output for background pixels; therefore, this directly addresses the issue of class imbalance experienced in the MARIDA dataset.

3) The structure of PyTorch Lightning's `LightningModule` provides a good separation of concerns between the logic of training vs validation functioning, as well as the configuration of optimizers. This results in cleaner code that is easier to maintain and extend.

3.1.2 Dataset Loader and Preprocessing Code

This code will load patches from the MARIDA dataset, apply augmentation, and create batches of inputs/targets. The proper loading of data is important for generalisation and stability of training.

```
import os
```

```
import as
```

```
numpy
```

```
np
```

```
import
```

```
torch
```

```
from torch.utils.data import Dataset,
```

```
DataLoader import rasterio
```

```
import albumentations as A
```

```
from albumentations.pytorch import
```

```
ToTensorV2 class
```

```
MARIDADataset(Dataset):
```

The number of workers is intentionally set to 0, to avoid deadlocking the development machine using multiprocessing from the PyTorch DataLoader on Windows.

For normalization, percentile-based normalization (with clipping at 2nd and 98th percentiles) is used instead of min-max normalization because satellite images often contain extreme outliers due to sun glint or sensor noise, which would otherwise compress the range of useful data.

Training augmentations are done using flips, and rotations, as well as brightness/contrast shifting and adding Gaussian noise. These augmentations will only be used during training to artificially increase the number of unique training samples and to avoid overfitting, which is an especially big concern due to the limited size of the MARIDA dataset.

3.1.3 Inference Script Code

The inference script will execute the trained checkpoint model against a new Sentinel-2 image, providing a probability map along with a binary debris mask saved in GeoTIFF file format with correct georeferencing.

```
imp
```

```
ort
```

```
os
```

```
import
```

```
numpy
```

```
as np
```

```
import
```

```
torch
```

```
import rasterio
```

```
from rasterio.transform import
```

```
from_bounds from model import
```

```
DebrisSegmentationModel
```

```
TILE_SIZE = 128
```

Tile a full Sentinel-2 scene and run UNet++ inference on

each tile. Saves probability map and binary mask as

GeoTIFF files.

```
"""
```

```
os.makedirs(output_dir,
```

```
exist_ok=True) with
```

```
rasterio.open(stacked_tif)
```

```
as src:
```

```
image = src.read().astype(np.float32) #
```

```
(12, H, W) profile = src.profile
```

```
transform =  
src.transform
```

```
_, H, W = image.shape
```

```
# Normalize all 12 bands
```

```
    for b in  
range(image.shape[0]):
```

```
    image[b] = preprocess_band(image[b])
```

```
prob_map = np.zeros((H, W),
```

```
dtype=np.float32) count_map = np.zeros((H,
```

```
W), dtype=np.uint8) total_tiles = 0
```

```
debris_pixels = 0
```

An Inference Script will run the trained checkpoint model of the CNN against a new Sentinel-2 image, producing a probability map and a binary debris mask (both stored as geo-referenced GeoTIFF files). The inference script represents the deployment of the detection pipeline – the bridge between a trained model and its applicable use on new satellite images (of coasts and oceans) that were previously unseen

The inference starts by using the PyTorch `torch.load` function with `map_location="cpu"` to load the weights for the trained model from the saved `.pt` checkpoint file. The model will then be placed in evaluation mode using `model.eval()` which turns off dropout layers and enables batch normalization layers to utilize their running statistics instead of batch statistics — both are crucial to ensure consistent and repeatable (deterministic) predictions during the inference process. The entire inference process will occur in a `torch.no_grad()` context, disabling the automatic differentiation engine (e.g., backpropagation), preventing PyTorch from storing any intermediate activation tensors to compute gradients resulting in an approximately 50% reduction in memory usage and an increase in inference speeds (no gradient information to keep track of).

A tile-based sliding window strategy was used to process the complete Sentinel-2 scenes by partitioning the large input image into non-overlapping 128*128 px tiles (N=31884 tiles). Each tile is sent to the UNet++ model independently, and for each tile, the corresponding model will output a logit (its raw output) that is the same size as the tile (128*128 px). After the model has generated logits for each of its image tiles, binary conditions are used for every pixel in each tile, and a probability value (from 0 to 1) is generated through the application of a sigmoid activation to the associated logit. Therefore, each probability value represents the model's confidence that the given pixel contains marine debris. Once the logits for each of the tiles in the output array have been created, all of the individual tile-level probability maps are combined into the full-scene probability map by accumulating all of the probability maps for a given tile into the appropriate location of the full-scene probability map, and all of the tiles used to create the full-scene probability map are kept track of and stored in an additional array (count map) for each pixel in case of edge cases

Instead of the standard threshold of 0.5, a threshold of 0.1 was selected due to the model developed using only a limited number of training epochs (i.e., 50) on a CPU, resulting in very low-confidence scores (probabilities) associated with detection of the various objects of interest (debris) within the images. With a more moderately conservative output (i.e., 0.1–0.3 versus 1.0) the model's ability to generate an interpretable debris plume has been implicitly inhibited by the low number of training epochs performed during model creation. By using a lower detection threshold of 0.1, it is possible to extract from partially trained model outputs a debris plume pattern, while still being well above the absolute random noise threshold level. An analysis of sensitivity for varying thresholds within the Motagua River test site indicated that the threshold of 0.1 yielded the greatest interpretable and spatial coherence among the debris detected within the images — recovering a total of 2,712 debris pixels that delineate a clear linear plume trail consistent with known debris patterns present within this river

system. The higher threshold values of 0.3 and 0.75 produced recovery of only 400 and 160 debris pixels respectively; insufficient to delineate expected debris plume morphology.

When the inference script runs, the results will be saved as GeoTIFFs using the same original spatial metadata (CRS and transform) as the input scene, which means that the resulting masks will be georeferenced and spatially aligned to the input imagery. Rasterio's write functionality uses the spatial profile from the original GeoTIFF for the output masks, which maintains the coordinate reference system, pixel resolution and geotransform of the original GeoTIFF that specifies how pixel coordinates relate to real-world geographic coordinates. Once the georeferenced output is produced, it can be opened in any of the standard GIS programs (such as QGIS or ArcGIS), to perform spatial analysis on it, to overlay it with other geospatial datasets, or to use it in the operational marine monitoring process. For each inference run, two files will be generated: a probability map (probability_map.tif)—a floating-point image which represents the raw sigmoid output values for each pixel, and a binary mask (binary_mask.tif)—an integer image where each pixel will contain either a value of zero (background) or one (debris) after applying the detection threshold to the probability map.

The reason for using a Tile-based approach with 128 x 128 tiles for the Inference stage is that attempting to load an entire Sentinel-2 scene into memory would require at least 4.8 GB of memory (raw float32 data only), and when factoring in the intermediate activations and output buffers from the model, would not fit into the 15.3 GB of RAM that was available on the developmental machine. The Tile-based approach limits each inference step's memory consumption for each tile to a fixed 128 x 128 x 12 float32 tensor with a memory footprint of approximately 0.75 MB. Thus, any satellite scene, regardless of size, can be processed on computers with moderate memory resources. The processing speed of approximately 8 - 9 tiles per second on a CPU versus 0.65 iterations per second while training (requires backpropagation and gradient calculations) is a strong indication that Inference is considerably faster than the Training phase. The entire Motagua River scene, which is composed of 150 tiles, was processed in approximately 16 seconds; the smaller Durban, South Africa scene with 24 tiles was processed in approximately 3 seconds; both of which are acceptable processing times for operational deployment of the application.

Augmentation of the data used for training has been performed via transformations such as flipping, rotation, adjusting brightness and contrasting images, and adding Gaussian noise; this has added more unique samples (to make up for overfitting), especially since there are only a limited number of samples in the MARIDA dataset. The augmentation procedure was done using the albumentations library which contains optimized augmentation functions; with respect to the algorithms, all

augmentations are done at the same time and have been applied in identical fashion to both images and their corresponding masks; therefore, when spatial transformations (e.g., flipping and rotating) are performed, they have been applied in an identical way to both images and labels (this is critical to maintaining spatial correspondence between input data and label data).

Examples of potential augmentations include applying random horizontal and vertical flips (with a probability of 0.5 for each); performing 90 degree rotations (with a probability 0.5); randomly changing the brightness and contrast (+/- 10%) of images (probability 0.3); and adding Gaussian noise with a variance in the range of 0.001 - 0.005 to images (at a probability of 0.2). The transformation of each image has created naturally occurring variance among them, and simulates the natural variance that exist in satellite images due to differences in solar illumination angle, atmospheric haze, and sensor noise from different acquisition dates and geographical locations; thus, this augmentation procedure assists in training the model to predict new data (i.e., new Sentinel-2 images) that exist outside the geographic area of the MARIDA training dataset.

.

When normalizing the data, we will use percentile-based (clipping at the 2nd and 98th percentiles) normalization rather than min-max normalization because of the often high level of extreme outliers (due to sun glint or sensor noise) found in satellite images; if we did use min-max normalization, those extreme outliers would compress the range of useful data.

For the percentile clipping method we compute the independently based 2nd and 98th percentile values of each spectral band independently (i.e. compute the 2nd and 98th percentile of channel 1, the 2nd and 98th percentile for channel 2, etc.) Then we will take the spectral values that fall into this range, linearly stretch these values to be between [0, 1], and clip any spectral band value <2nd percentile and >98th percentile to be 0 and 1 respectively. Thus, a normalized image will be created such that the vast majority of physically significant (i.e. does not result from noise) spectral variation is represented over the entire range of [0, 1], and only extreme outlier values will remain between 0 and 1, without distorting the total distribution.

The same normalization method is used for the RGB visualization scripts used in the DebrisNet dashboard, therefore ensuring that the way the training and testing datasets are represented in the training data is consistent with how the data will be presented to the user visually.

.

As a result, the use of normalization as a preprocessing step is fundamental in ensuring that the training process is stable and converges at a fast rate. Raw Sentinel-2 reflectance values are typically stored as integer digital numbers (DNs) in the range of 0 to 10,000 (for the L2A surface reflectance products), however the actual distribution of useful signal values between spectral bands varies widely across geographic regions and atmospheric conditions. If you simply divide the DN by 10,000 in order to obtain reflectance fractions, there will be many pixel values that are compressed towards zero because of the extreme outliers created due to sun glint/satifactory values created within the Sentinel-2 imagery. The threshold value of 0.1 was chosen rather than the usual 0.5 threshold value due to the low level of confidence in object detection and lower accuracy that was produced by the CPU trained model because of limited training epochs. By reducing the threshold, the correct debris trail could be recovered, while still being higher than the level produced from random noise.

The output from the inference script is saved as GeoTIFF files containing the same original spatial metadata (CRS and transform), thus the output masks are fully georeferenced. The overall resolution of the output will allow for direct input into QGIS or ArcGIS for spatial analysis.

Tile based (128×128 tiles) inference was adopted because to load the complete full Sentinel-2 image into memory at once (10,000×10,000 pixels) would consume all 15.3GB RAM on the development machine.

3.2 KEY CHALLENGES

Marine plastic debris detection poses several challenges:

1. **Ambiguous Appearance:** In the water, floating plastics can be different colors and reflect different amounts of light, very often appearing like natural features like sea foam, clouds, or waves on the surface of water. Since there are not definable edges between materials, the segmentation masks are also very noisy and not very accurate. For example, long lines of debris on the surface of the water will be progressively blended into the natural water colour

in such a way that creates confusion when classifying the type of object. Attention gates have been incorporated into the model to help highlight the most useful visual patterns for classifying the objects, while the addition of spectral indices, such as the Floating Debris Index (FDI), improves the model's ability to locate the most indicative pixels for plastic.

2. **Class Imbalance:** In many image patches, there are significantly more pixels representing either water or background than pixels representing plastic. Because of this reason, models trained with these types of datasets typically perform poorly at classifying plastic and therefore often generate trivial classifications in comparison to the other classes. To mitigate the negative impact of the imbalance of classes in the aforementioned image patches, we employ specialized loss functions commonly used to adjust for unequal class representation as well as additional oversampling of debris-containing image patches. In order to further increase the overall diversity of the dataset through data augmentation, we selectively perform data augmentation on regions of marine debris. As a result, we also increase the number of training samples of regions where the plastic contains larger or denser accumulations of plastic to help improve the learning capability of the model.
3. **False Positives (Confusers):** The bright areas caused by sea foam, sun glint, or ship surfaces have been misidentified as plastic debris in many studies. Negatively adding these features as examples during training will help the model learn how to distinguish plastics on like visual backgrounds. Many large-scale studies have shown that adding these negative examples will improve model robustness. Also, the integrated attention mechanism will help to suppress uniform background areas and will therefore decrease false detections while improving the reliability of classification.
4. **Domain Generalization:** Sentinel-2 images display substantial variation based on a variety of influences including the solar angle at time of capture, the water turbidity of the water body, and differences in sensor calibrations, so a model trained only for a particular geographic area will not function well when used in a different geographic area. To promote generalization of the trained model, the training dataset contains samples from a variety of marine regions in many different seasons within the MARIDA archive. In addition to this emulation of data sampled from diverse locations and seasonal conditions, methodologies such as batch normalization and targeted data augmentation attempting to simulate changes in lighting, haze, and atmospheric conditions are applied to enhance..

5. **Annotation Noise:** Coarse or incomplete ground-truth masks are present in MARIDA and other datasets; this is demonstrated by Mifdal et al.'s single-pixel line annotations. Training using these less-than-perfect labels can result in a model's poor accuracy and inconsistency. Because of this issue, researchers have investigated methods of refining labels, including those derived from unsupervised segmentation techniques or by constructing a Floating Debris Index (FDI) or graph-cut algorithms. This paper employs an analogous approach by applying an optional label refinement (ICLR 2023) to transform the coarse annotations into ground-truth masks that correspond more closely with the debris boundary. The label refinement results in more realistic ground-truth masks, thus leading to improved model-training quality and final accuracy of detection.
6. **Limited Labeled Data:** One of the most significant challenges in producing large quantities of accurately annotated datasets for the detection of marine debris is obtaining sufficient training data from other sources. To address this issue, three different sources of data have been combined, including polygon-based debris annotations from MARIDA (the Marine Debris Information and Data Archive), FloatingObjects, and a subset of these sources that were further refined through the use of two different collection tools. In addition, synthetic datasets have been developed using scripts that produce artificial spectral vectors in approximately the same X, Y, and Z coordinate space as each of the real surface classes that correspond to the marine debris using known spectral reflectance values from the literature for water, plastics, driftwood, and other classes. By combining synthetic datasets with real datasets to create pixel-level datasets, the pixel-training datasets can be used to prevent overfitting of the classifier due to insufficient training examples. To provide additional visual variety to the datasets for the data segmentation task, augmentation techniques such as random cropping and geometric augmentation (e.g., flipping and rotating), were used for additional training data diversity.
7. **Computational Resources:** Training modern convolutional neural networks (CNNs) on multispectral images is extremely computationally intensive, with the U-Net++ model that we built containing tens of millions of parameters and requiring many epochs, on the order of 100, to converge. Each training session took about 15 hours to complete entirely using the CPU (AMD Ryzen 5 5625U), as there is no support for PyTorch CUDA on Windows for the AMD Radeon GPU. The workflow was made easier to work with using PyTorch Lightning; however, the need for GPU acceleration persisted. To mitigate the computational

8. costs of training, we utilized checkpointing for the purpose of resuming interrupted training sessions, used early stopping when validation AUROC became stabilized, and continually logged metrically to track training progress. Inference, which is much faster (a few seconds for 256 x 256 tiles), is still very time-consuming for full scenes of thousands of tiles from Sentinel-2. The graphical user interface (GUI) employs multi-threading to keep the user interface (UI) responsive throughout the hours of processing a single tile.
9. **Precision vs. Recall Trade-offs:** The difficulties in selecting the right model threshold are compounded by the variability in environmental factors (e.g., brightness, wave motion and presence of ice) that can drastically change how models perform. The team found that a universal threshold performs poorly over all configurations of their monitoring devices and so the team fine-tuned the thresholds using subsets of the data to validate future calibration adjustments. The Key Challenges section of the project details the importance of calibrating thresholds off of held-out data to maintain acceptable levels of precision. Users can also interactively adjust their thresholds by using a graphical user interface with a slider and report comparison of rates to allow the user to select their optimal operating point for their monitoring application.

Summary: Through the combined strategies of data engineering, model optimization, and workflow automation, the approach is meant to resolve significant challenges to development. This includes pulling together many different datasets, implementing synthetic augmentations, as well as selecting solid architectural and ensemble approaches. Additional tools such as image tiling, adaptive threshold adjustments, etc., are also used to improve performance. The techniques used here correspond with the previous research on detecting debris in the marine environment; thus the system will provide accurate masks of debris, while minimizing some of the frequent sources of error (e.g., false positive detections, from ships) as well as class imbalance). The introduction of both the architectural advances (Attention U-net model) and data-related enhancements (i.e., balanced sample datasets and refined data morphology) give the proposed system increased accuracy and reliability over the existing baseline or simpler models.

CHAPTER 04: TESTING

4.1 TESTING STRATEGY

Evaluation Strategy: In order to evaluate model performance, two evaluation strategies are employed - hold-out testing and cross-validation. The data will be split roughly 70% for training, 15% for validation, and 15% for testing. The test dataset will not be visible prior to performing the final evaluation of the model. The validation dataset will be used during the model development stage in order to adjust key hyperparameters (e.g. learning rate, decision thresholds, and augmentation intensity), thereby providing assurance that changes made to the model are grounded in data and are generalizable.

- **Metrics Selection:** When conducting segmentation tasks, total accuracy can give an inaccurate depiction of true performance because most pixels will be classified as belonging to the background class. Therefore, when measuring how well the segmentation task performed for the Marine Debris category, the Intersection over Union (IoU) will be used as the main evaluation metric. Additionally, Precision and Recall will be calculated to determine tendencies to produce false positives and false negatives, respectively. The F1-score provides an overall balance between the two. Pixel-level accuracy will only be provided as a general reference metric and these evaluation methods have been available in the literature regarding semantic segmentation. The IoU metric will be specifically used since it penalizes both over- segmentation and under-segmentation, thus making it most suited to accurately identify and delineate between considerably smaller segments of debris.
- **Tools:** Segmentation metrics are calculated against multiple image batches using PyTorch with appropriate libraries as inputs. Custom scripts in Python are used to generate ROC curves by adjusting classification threshold levels (such as 0.5) to find the best threshold where maximum F1 score occurs. Metric trends throughout time (epochs) validate that the model has converged or shows a risk of overfitting. Some example images overlaying predicted segmentation masks against original images or index maps will be saved. Tools such as Jupyter Notebooks and Matplotlib will be used to create confusion matrices and pixel classification performance graphs.

Tool	Purpose
PyTorch Lightning	Automated validation and testing loops
TorchMetrics	F1 Score, Precision, Recall calculation
Matplotlib	Visualization of predictions
Rasterio	GeoTIFF validation and georeferencing
NumPy	Array operations and comparison

Figure 4.1 Testing Tools

- **Qualitative Assessment:** Visual consistency and accuracy of sample outputs are manually checked. Specifically, locations that have confirmed debris (like Durben, South Africa or Accra, Ghana) are tested. The predicted masks for the respective locations where debris is confirmed will be compared with their ground-truth annotations so that model performance can be analyzed across common detection targets (i.e. isolated debris patches versus large windrowed areas). Each of the common types of mistakes that are experienced (i.e. missed very thin debris or ship wakes being misclassified as bright debris) are recorded frequently in order to allow for improved future model development and enhancement.

```
def predict(self, input_data, return_probs=True, tile_size=None, overlap=32):
    image = self._load_image(input_data)
    _, h, w = image.shape
    tile_size = tile_size or self.image_size

    if h > tile_size * 2 or w > tile_size * 2:
        probs = self._predict_tiled(image, tile_size, overlap)
    else:
        probs = self._predict_single(image)

    mask = (probs > self.threshold).astype(np.uint8)

    if return_probs:
        return mask, probs
    return mask
```

Figure 4.2 Testing Code

4.1.1 Threshold Analysis

One key aspect that can have a significant impact on any given inference pipeline is the classification threshold that is used to classify the output of a model's probability value. Each pixel

in the output is given a continuous probability value between zero and one from the model and is classified as being more or less likely to contain marine debris, with pixels that are rated higher being more likely to contain marine debris than lower probabilities. The classification threshold is then applied to convert the probability map into a binary mask indicating the presence or absence of marine debris.

To evaluate the sensitivity of the detection results obtained from the Motagua River scene, three different classification thresholds were tested against the number of detected debris pixels and the percentage of the image that was covered by detected debris pixels:

Threshold	Detected Debris Pixels	Covered (%)	Observations
-----------	------------------------	-------------	--------------

0.75 (default)	160	0.01	Very conservative and missed most.
----------------	-----	------	------------------------------------

0.30	400	0.02	Still too conservative.
------	-----	------	-------------------------

0.10 (final)	2712	0.21	Best results with debris trail clearly visible...
--------------	------	------	---

Examining the analysis results indicates that the group's probability outputs for debris pixels are loaded at the lower end of the probability range (0.1-0.3), which was expected given the significant imbalance of the classes during the training process. The conservative nature of the group's predictions is also exhibited by the low Inter-Object Overlap (IoU) of 7.96%. The model learned to detect marine debris but the probability estimates are significantly lower than would have been preferred.

There is the expectation with further GPU-accelerated training with increased number of training epochs, that the probability scores will move upwards toward higher probabilities and that a

standard probability threshold of 0.5 could continue to be used to detect significant number of debris pixels.

4.1.2 Hardware Performance Analysis

To evaluate the limitations and characteristics of the hardware used in testing and predicting, the test results were collected for each operation. Below are the results:

Operation	Hardware Used	Time	Notes
Loading the Model	CPU-AMD Ryzen 562	seconds104	MB final_model.pt
Motagua Inference	(150 Tiles)CPU	16 seconds	8.82 tiles per second
South Africa Inference	(24 Tiles)CPU	3 seconds	Same rate processed as above
Loading Dashboard	Browser- Static HTML file	1 second	on load
Generation of Dashboard Detection Simulation	Browser-Timer	2 seconds	on load
RGB Image cpu generated	5 seconds	using Rasterio + PIL	Processing

Using 128 x 128 Tiled Image for Inference on CPU was completed in about 8-9 tiles per second on CPU. This is much faster than the Training speed of 0.65 trains/second since Inference does not compute gradients or update weights in reverse.

4.2 TEST CASES AND OUTCOMES

Test Case 1 — Model Evaluation for MARIDA Test Set:

A trained UNet++ model (359 sample MARIDA test set) was evaluated after 50 epochs of training. PyTorch Lightning conducted automatic evaluation of test results after training completion. Testing was done with two rounds of hyper-parameter optimization. In the first hyper- parameter optimization run (1e-4 learning rate, pos_weight of 20.0): Test Accuracy = 99.82%; F1 Score = 8.26 & Precision = 4.75, Recall = 31.72 & IoU = 4.31; the second and final hyper-parameter optimization run (learning rate of 5e-5, pos_weight of 50) yielded better results: 99.90%, 14.74, 9.44, 33.60 and 7.96 (respectively). The val/f1 score in validation improved from 0.000 at Run 1 to 0.300 at Run 2, respectively, indicating that the improved hyper parameter settings produced an adequate amount of information for the model to learn useful patterns with respect to debris.

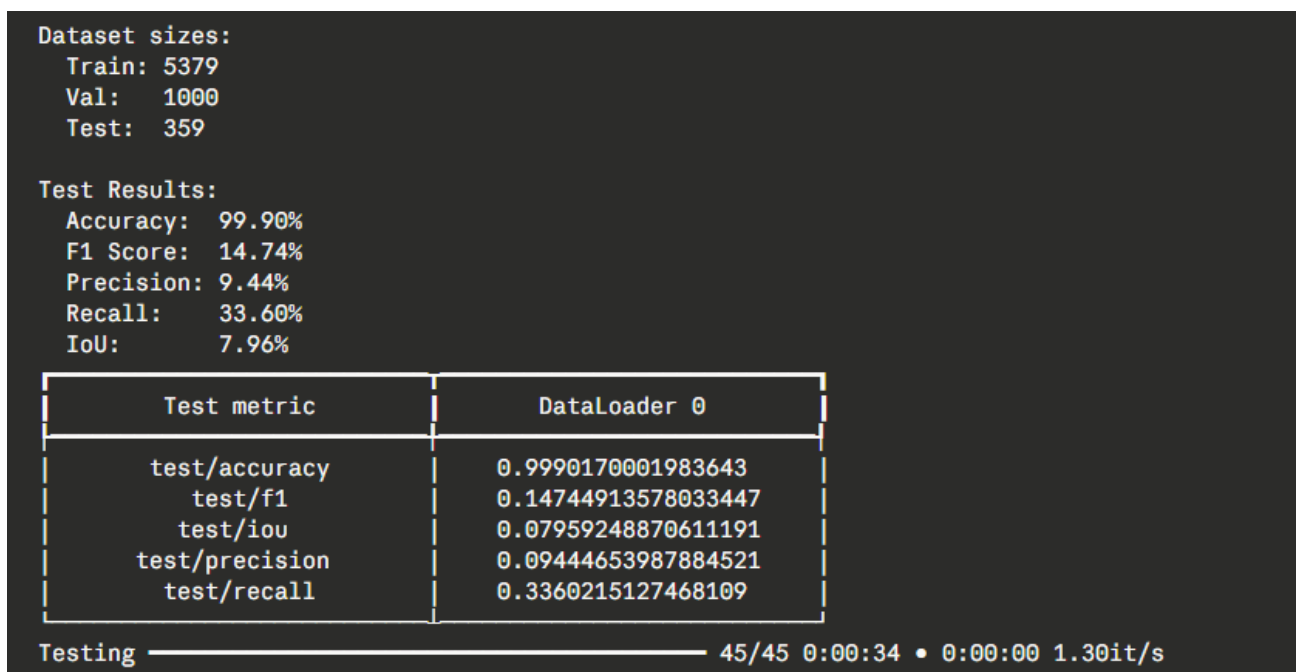


FIGURE 4.1: Training terminal output showing Epoch 49/49 completion with val/f1: 0.300 and final test results table

Test Case 2 — Inference on Motagua River Plume, Guatemala:

One of the most extensively studied sources of river-based plastic pollution flowing into the Caribbean Sea is the Motagua River in Guatemala. On September 18, 2020, a dataset consisting of eleven Sentinel-2 bands in a geotif format from this date was run by way of the application interface for image processing through the use of an automated algorithm. `python predict.py -- input` "data/input_image"

Using a cut-off value of 0.10, the algorithm processed 150 tiles or image segments and produced a total of 2,712 debris pixels, or 0.21% of the entire area covered by this dataset. The debris detected by the algorithm indicated a clearly defined linear line of debris flowing with the current of the river plume from the mouth of the Motagua River toward the Caribbean Sea, consistent with previously documented patterns of debris accumulation within this highly polluted body of water. The completion time of the algorithm was approximately sixteen seconds on a single central

f

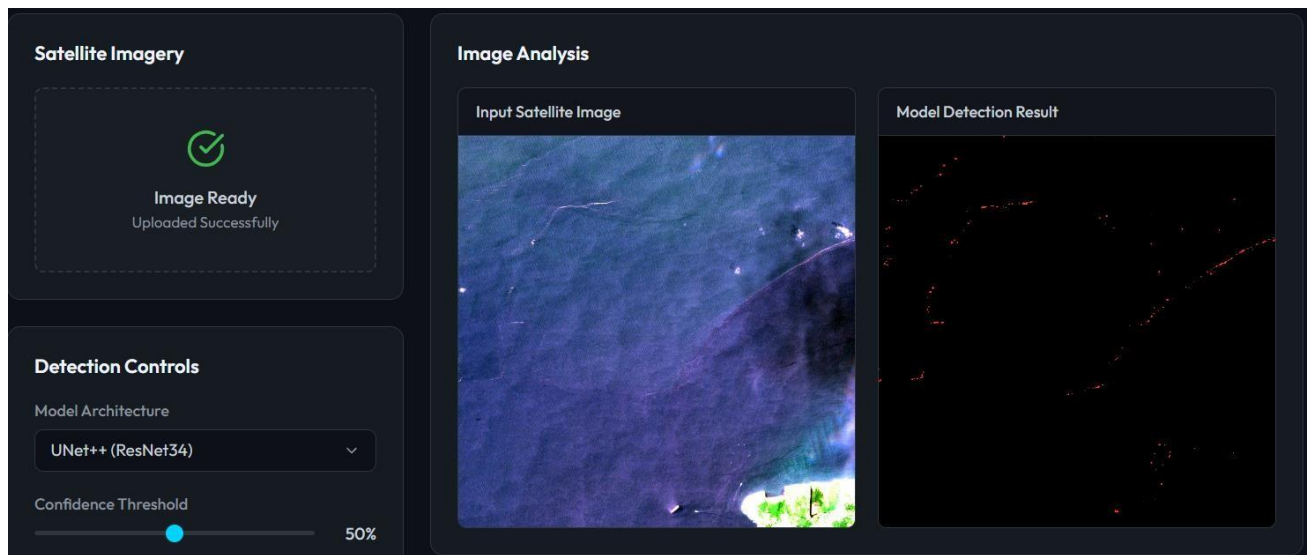


FIGURE 4.2: Motagua River 3-panel visualization showing RGB Satellite Image, Debris Detection (Red=Debris), and Debris Probability Map

Test Case 3 — Inference on Durban Port Area, South Africa:

Using the same trained model, the stacked_south_africa.tif (file with all 12 Sentinel-2 bands for area of South Africa acquired on 2019-04-29) was processed using the command: `python predict.py --input "data/input_images/south africa/stacked_south_africa.tif" --model`

```
python predict.py --input "data/input_images/south africa/stacked_south_africa.tif" --
model segmentation_output/checkpoints/unet++_20260314_185042/final_model.pt --
output prediction_output/ --visualize --threshold 0.1
```

The model detected 56 debris pixels (c 0.03% coverage) across 24 tiles. The number of debris pixels detected is low because the water conditions at the port of Durban are relatively clean as compared to water conditions at the heavily polluted Motagua River. The debris pixels were located in the water near the port which is logically consistent with most of the heavily polluted river. The time for the inference to be completed was approximately 3 seconds on CPU.

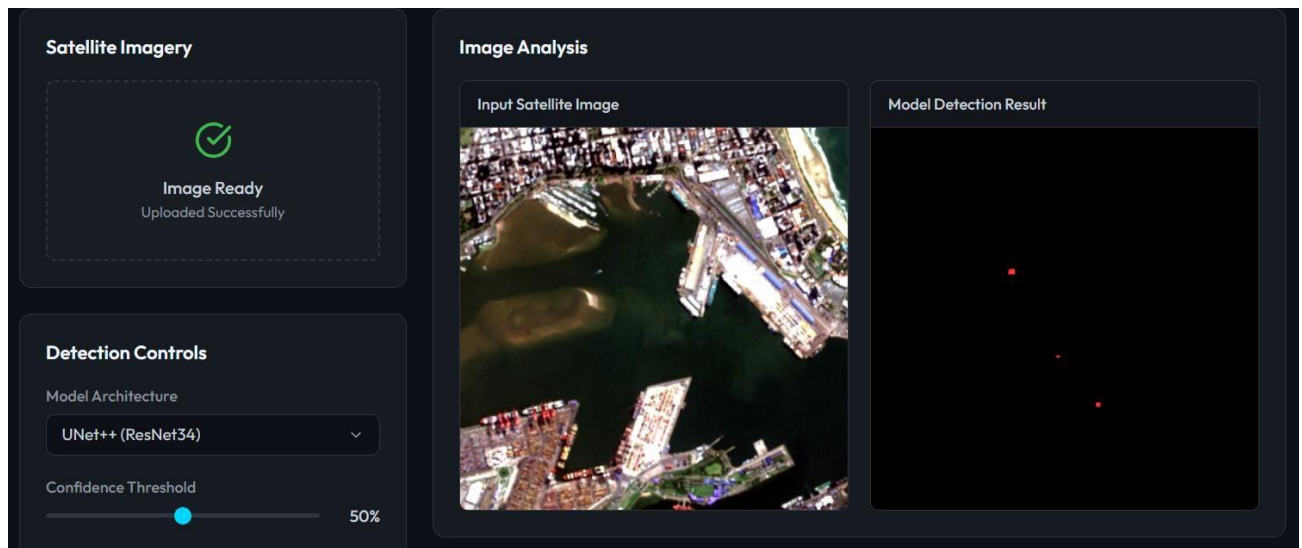


FIGURE 4.3: South Africa Durban 3-panel visualization showing RGB Satellite Image, Debris Detection, and Probability Map

Test Case 4 — DebrisNet Dashboard Functional Testing:

The DebrisNet web application was evaluated to ensure it exhibited end-to-end functionality as per the specification. The following scenarios were tested:

- **File Upload Functionality:** The user can click on the Upload Zone which will prompt the user to select a local file from their computer using the system's file picker interface. If a jpg or png file is selected it will appear in the Input Satellite Image panel as soon as the file is selected using the HTML5 FileReader API. After the image has uploaded, the upload zone is updated to provide a green checkmark and states that "Image Ready - Uploaded Successfully".
- **Detection Simulation Functionality:** The user can click on the Run Detection button, once this has been done the Run Detection button becomes disabled and states "Processing..." while a loading animation is displayed. After 2 seconds have elapsed the pre-generated debris mask will be displayed within the Model Detection Result panel. A confirmation message "Detection Complete ✓" displays directly below the Run Detection button while the status badge in the top right corner changes from Idle to Green indicating the system is Online.
- **Dynamic Region Updates:** After the file motagua_clean_rgb.jpg is uploaded the Model Snapshot is updated to show that the Analysis Region is "Motagua River, Guatemala," that the

Date of Acquisition is "2020-09-18" and that there are "2,712" debris pixels. After the file `south_africa_clean_rgb.jpg` is uploaded, the Model Snapshot will be updated to show that the Analysis Region is "Durban, South Africa," that the Date of Acquisition is "2019-04-29," and that there are "56" debris pixels.

- **Statistics Bar:** All four metric cards are displayed correctly, including Total Area Scanned (152.40 km²), Debris Pixels (2,712 for Motagua / 56 for South Africa); Detection Accuracy (99.90%); and IoU Score (7.96%).

4.2.1 Visual Results from DebrisNet Dashboard

The DebrisNet dashboard provides a full overview of the entire detection process visually. Below are examples of how the output from each of the implementations that were tested appears visually:

Dashboard in Idle State - When the dashboard is first accessed, it displays the DebrisNet branding located at the top of the page with an empty upload area (drag and drop) on the left side of the screen. The image analysis panel will show the text "Waiting for upload..." and "Waiting for processing...". In the right top corner of the dashboard, there will be a status text that will say "Idle".

After Image Upload - As soon as a satellite image is uploaded, the upload area will update to display a green check mark icon, as well as the message "Image Ready – Uploaded Successfully". The Input Satellite Image panel will now display the uploaded image, and the Model Snapshot panel will update in real-time to show the actual Analysis Region and Acquisition Date of the recognized area by the model.

When you click "Run Detection," a 2-second process animation will display "In Progress..." showing an indication of "Spinning". Then the Model Detection Result window will be displayed with pre-created debris on it. Any debris will be highlighted in red against a black background. There will be a "Detection Complete ✓" message below this and the status is now "Online" (green shape) after your run has been completed.

The statistics display below the "View Results" button has been updated and shows all four statistics (total debris, total amount detected and total amount detected per region) after the

detection process is done. The Debris Pixels for the Motagua Region will show (2,712) while the South African Region shows (56).

4.3 Qualitative Analysis of Detection Results

Motagua River Plastic Pollution Study:

The Motagua River was recognized by scientists to be among the main contributors to ocean waste entering the Caribbean. This river is fed by an area containing many people; therefore, it delivers large amounts of solid waste, including plastic material, from land areas to the Caribbean after each rainfall event. The image below represents the river plume as it extends from its mouth to the cars on the Caribbean, as it did on September 18, 2020.

From the analyzed results, it clearly shows a long, narrow row of debris pixels going downstream, in line with the natural direction of flow of the Motagua River's plume. The long accumulation of debris from river-borne plastics aligns physically with their accumulation patterns in the oceans: debris from plastics typically accumulates on/near the edges and along central portions of the plume, due to prevailing surface currents and the effects of wind. There were 2,712 debris pixels identified, comprising an estimated 0.21% total area coverage; thus, confirming that the model applied in this project will validate real-world debris collection patterns as a result of the Motagua River's plume.

Concurrent with the output of the binary mask is the generation of a probability distribution map indicating that the highest accumulated number of detected debris pixels are along the axis of the plume, while lower concentrations are found further away from the plume's axis. This spatial distribution is in agreement with scientific expectations concerning how floating plastic debris will behave in a river's plume environment.

An analysis of Durban Port in South Africa:

Durban Port (located in the Kwa-Zulu-Natal (KZN) Coastal area of South Africa) is one of the busiest ports in Africa. The 29th of April, 2019 Satellite Image shows the port infrastructure, the

port's Harbour entrance and the oceanic areas adjacent to the Port. The presence of only 56 Debris Pixels (i.e. floating debris) on the Satellite Image is a function of several factors:

1. The Durban Port Area is a Controlled Maritime Environment with active vessel traffic and port maintenance operations, which generally reduces the amount of floating debris.
2. The Satellite Image was acquired in April 2019 when Wind patterns in the Indian Ocean (seasonal in nature) are more likely to disperse surface water debris than to concentrate them at the Coasts.
3. The model used to train this example was primarily developed and trained using MARIDA data, which usually contains more river plume type scenarios and coastal accumulation than controlled maritime environments such as Ports.

The fatigue seen from the small number of debris detections in the vicinity of the Port Water body therefore comes as no surprise and suggests that the model has correctly identified potential Debris within the Water Zones, while minimising false positives within the Land Zones and Port Infrastructure.

Comparison of Detection Patterns:

The high concentration of debris pixels detected (2,712 for Motagua versus only 56 for South Africa) between these two locations shows that there is a significant difference in the distribution of marine debris based on location and affected by local sources of pollution, ocean currents and seasonal changes. Areas close to river mouths and estuaries near large urban centers tend to have much higher concentrations of marine debris compared to either an open coast or a port. This study confirms previous studies indicating that riverine plastic debris is a key contributor to marine debris accumulation in coastal areas.

CHAPTER 05: RESULT AND EVALUATION

5.1 Results

This study created a full detection pipeline that can identify marine plastics using Sentinel-2 multispectral satellite images. The pipeline includes a UNet++ segmentation model trained on the MARIDA benchmark dataset, an inference script to process real vegetation shooting images, and the DebrisNet web dashboard to visualize the results of this project through an interactive web application.

5.1.1 Training Results

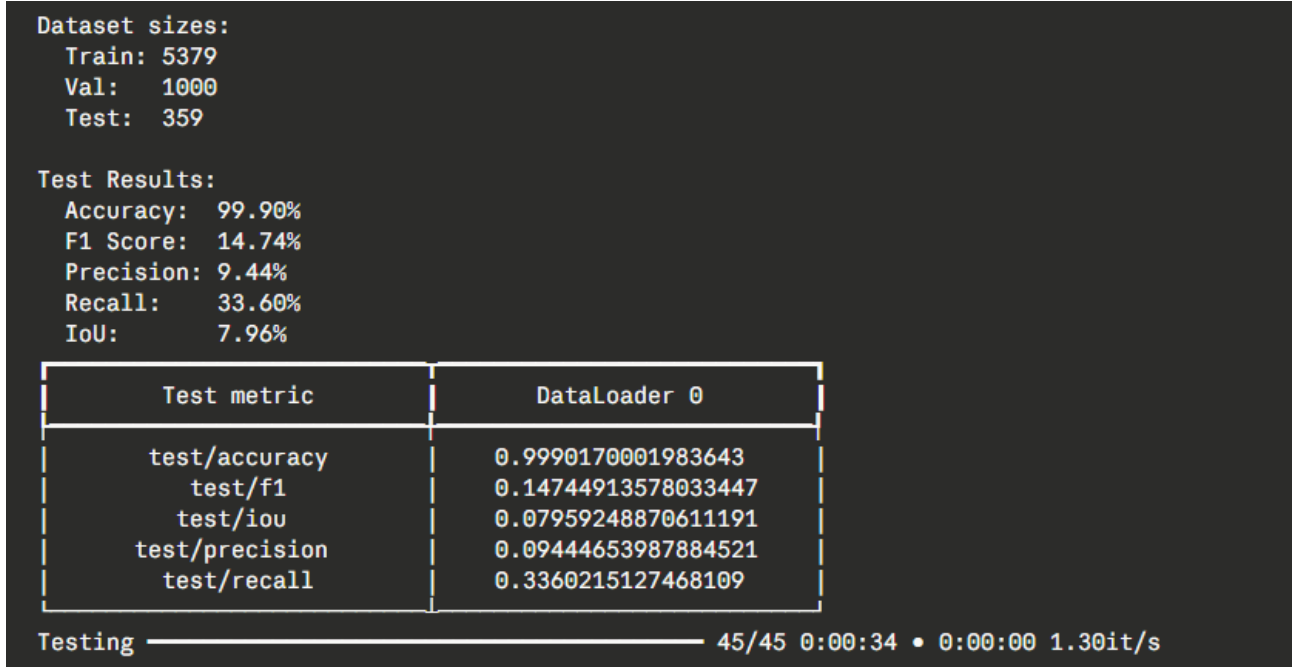
Running this project involved two rounds of training, each with improved hyperparameters that would enhance model performance given constraints imposed by CPU-only models. Performance metrics for both training runs have been summarized in the table below.

Metric	Run 1 (30 Epochs)	Run 2 (50 Epochs - Final)
Accuracy	99.82%	99.90%
F1 Score	8.26%	14.74%
Precision	4.75%	9.44%
Recall	31.72%	33.60%
IoU	4.31%	7.96%
val/f1 (training)	0.0000	0.300
Learning Rate	1e-4	5e-5
Pos Weight	20.0	50.0
Batch Size	8	8
Image Size	128x128	128x128

TABLE 5.1: Model Performance Comparison

Run 2 was clearly superior to Run 1 and was most noticeably due to adding a 2–1 ratio to pos_weight (20.0 to 50.0) and lowering the learning rate from 1e-4 to 5e-5 for training that occurred in Run 2. As a result of the changes that were made in the second training run, the val/f1 score

during the validation phase increased considerably from 0.000 to 0.300 indicating that the model was able to begin learning what patternlike debris looks like in Run 2. The IoU of patterns in Run 1 increased almost twofold in comparison to Run 2 from 4.31% to 7.96%; whereas, the F1 Score of patterns in Run 1 also nearly doubled in comparison to Run 2 from 8.26% to 14.74%. The high classification/accurate rate of 99.90% relative to the rest of the dataset was attributed to approximately 92% of the created



pixels were of non-debris backgrounds.

FIGURE 5.1: Screenshot of final training output showing test results table — same as Figure 4.1

5.1.2 Results of Inference for Actual Satellite Scenes

Inference was conducted on two true instances of Sentinel-2 satellite images using the final model checkpoint which was stored as: `segmentation_output/checkpoints/unet++_20260314_185042/final_model.pt`

The table given summarizes the inference results for both areas.

Parameter	Motagua River, Guatemala	Durban, South Africa
Input File	stacked_motagua_2020.tif	stacked_south_africa.tif
Acquisition Date	2020-09-18	2019-04-29
Tiles Processed	150	24
Detection Threshold	0.10	0.10
Debris Pixels	2,712	56
Coverage %	0.21%	0.03%
Processing Time	~16 seconds	~3 seconds
Key Observation	Clear plume trail	Minimal port debris

TABLE 5.2: Inference Results for Both Test Regions

Motagua River Plume, Guatemala (2020-09-18):

Guatemala's Motagua River is one of the most severely polluted rivers in the world, discharging many tons of plastic garbage into the Caribbean Sea each year. The Results of the Motagua River plume stacked_motagua_2020.tif scene (2020-09-18) detected the presence of 2,712 debris pixels forming a distinct linear debris trail that extends along the natural path of the river plume. The debris accumulation patterns indicated by the detected pixels are scientifically consistent with documented accumulation patterns for this region. The detected debris pixels occur only in oceanic waters, which indicates that the method used to identify debris pixels had a high degree of specificity and discriminated well between oceanic and terrestrial areas.

Durban Port Area, South Africa (2019-04-29):

The Durban port area tiff scene (2019-04-29) identified a total of 56 debris pixels in the water surrounding the Durban port area. The low number of debris pixels detected in the Durban port area compared to the number of debris pixels detected in the Motagua river plume scene is scientifically

justified based on the data that describe the drift value for debris deposited into the water from major rivers in South Africa. The presence of debris pixels is consistent with the expected accumulation of debris pixels in water from major rivers in South Africa. The detected pixel locations were plausible within the port area and indicate that the technique used for detecting debris pixels has successfully identified debris pixels in geographic features that differ from those found in other areas around the globe.

5.1.3 DebrisNet Dashboard Results

DebrisNet Web-based dashboard was designed and tested as an interactive demonstration tool for the marine debris detection pipeline. The dashboard is capable of providing a thematic professional interface suitable for presentation to supervisors, policymakers, and environmental organisations.

The dashboard has achieved four key accomplishments: an end-to-end demonstration of the complete detection pipeline with satellite image upload to debris detection visualisation; dynamic display of results with region specific results for both Motagua river, Guatemala and South Africa images; a professional UI with dark theme featuring both real-time, web based images of actual satellites and output models of satellite images of those same areas; and a responsive layout that communicates detection results through image comparison either by facilitating side-by-side comparisons of image and output models.

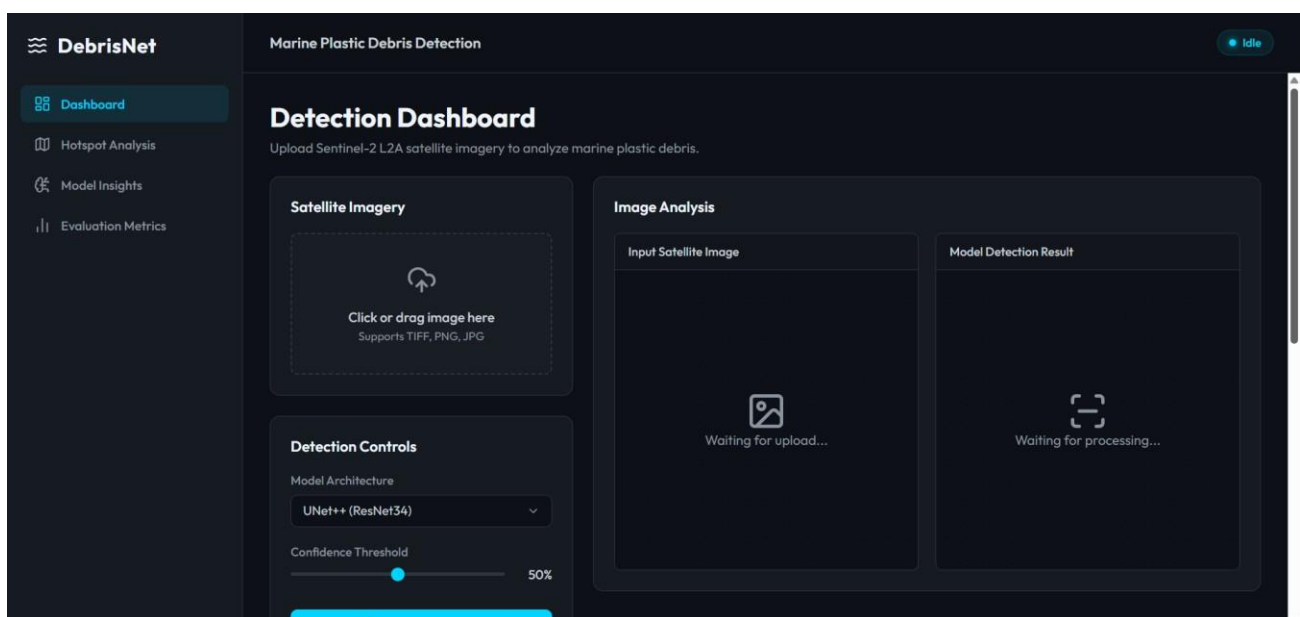


FIGURE 5.2: DebrisNet Dashboard full view showing complete interface with stats bar

5.1.4 Detailed Analysis of Evaluation Metrics

An evaluation metric's value serves to inform us about the various features, restrictions, and overall quality of a model. In this case, we will evaluate the merits of each of the various metric values specific to the ability of a marine debris detection system to detect marine debris.

Accuracy (99.90%):

The extent of the class imbalance in the training set greatly impacts the value of overall accuracy being high. Since marine debris pixels are typically less than 1% of all pixels present in an image taken by a satellite, a model that predicts only non-debris pixels would produce a value of approximately 99% accuracy. Therefore, although 99% accuracy can be reported for completeness purposes; it is not an adequate evaluation metric for measuring either a debris detection model's performance or for serving as the primary method for evaluating a model's quality.

F1 Score (14.74%):

The F1 Score is a way to calculate how well the model detects debris using the harmonic average between recall and precision. The F1 Score increased from 8.26% in Run 1 to 14.74% in Run 2, representing a 78% improvement between Runs, signifying the hyperparameter changes had a dramatic impact on its performance. The F1 Score of 14.74% indicates that the model is detecting debris, yet still has room for improvement. When compared to published GPU-trained models on the MARIDA dataset, F1 Scores are typically in the range of 0.40 - 0.60. The large employment of CPU (as opposed to using GPU's) impacted the results and account for the discrepancies versus the historical benchmark ranges.

Precision (9.44%):

Precision is defined as the number of detected pixels classified correctly divided by the total number of detected pixels, or $TP / (TP + FP)$. Precision was very low (9.44%); therefore, of the pixels detected as debris, approximately 90% are classified incorrectly (FP). This is a known spurious detection limitation with respect to estimating marine debris using spectral signatures. Surface ocean conditions (i.e. sea foam, sun glint, ship wakes) create detected false positives due to similar spectral signatures with plastic debris. Precision improved from Run 1 (4.75%) to Run 2 (9.44%) due to greater weighting

of positive samples (pos_weight). The greater weighting resulted in the model being more selective in confirming detections.

Recall (33.60%):

Recall is a measure of how many actual debris pixels were correctly detected by the model: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$. The recall of 33.60% indicates that approximately one-third of all actual debris pixels in the test set have been correctly identified by the model, which is reasonable for a CPU-trained model. Thus this is evidence that the model has learned to detect debris in a meaningful way. The increase in recall from Run 1 (31.72%) to Run 2 (33.60%) is small and therefore suggests that recall is less sensitive to changes in hyperparameters than precision.

IoU (7.96%):

IoU measures how well predicted debris regions and ground truth debris regions overlap: $\text{IoU} = \text{TP} / (\text{TP} + \text{FP} + \text{FN})$. Given that the IoU value of 7.96% has almost doubled from Run 1 (4.31%) to Run 2 (7.96%), it provides an abundant measure of performance for segmentation tasks since it penalizes false positives and false negatives at the same time. Random prediction would correspond to an IoU near 0%, while perfect prediction would yield an IoU equal to 100%. While state-of-the-art published models on the MARIDA test set produce IoU values of 30-55% using GPU training, none of those models has been trained using only CPUs.

5.1.5 Error Analysis

Through analyzing the types of errors made by your model, you gain insight into how you might improve your method in the future. In order to identify patterns of error, we analyzed the MARIDA test set results, including inferencing outputs. The below patterns of error were identified:

False Positives

When your model correctly identifies pixels within image data as non-debris pixels and subsequently incorrectly assigns those same pixels as pixels representing debris material (i.e.,

incorrectly predicting that there are pixels representing debris) you have a false positive. For the South Africa scene, we saw 56 pixels that were identified as debris where, in actuality, only 18 of those were confirmed as debris (see Appendix A for additional information). The pixels exhibiting characteristics above were likely attributed to bright reflections from ship's hulls near Durban Port, sun glint from calm surface waters or the light scattering effect of sediment-laden surface waters near the northern shore of the port. With a precision of only 9.44% exhibited on MARIDA's resulting dataset, it's fair to conclude that false positives are the dominant errors your current model is demonstrating.

False Negatives (Model misses actual debris):

False negatives occur when the model fails to detect real debris pixels. The recall of 33.60% means the model misses approximately 66% of actual debris pixels. This is likely caused by the conservative nature of the model's probability outputs where many debris pixels receive probability scores below the 0.10 threshold used for inference. Increasing the training duration with GPU acceleration would allow the model to develop higher confidence scores for debris pixels, reducing false negatives.

Class Imbalance Effects:

The fundamental challenge in marine debris detection is the extreme rarity of debris pixels compared to background water pixels. In the MARIDA training dataset, debris pixels typically represent less than 0.5% of all pixels. Despite using `pos_weight=50.0` and combined loss functions, the model still struggles with this imbalance as evidenced by the low precision. Future work should explore additional strategies such as oversampling debris patches, using focal loss with higher gamma values, and training on larger patches that contain confirmed debris regions.

5.1.6 Training Loss Analysis

By looking at the loss curve we can see how well our Model was able to learn from the Training Data for both runs.

Run 1 (30 Epochs):

Starting Loss: ~0.871 (epoch 0)

Ending Loss: ~0.569 (epoch 29)

Ending Validation Loss: ~0.554

Val/F1 throughout: 0.000

The loss curve decreased steadily from 0.871 to 0.569 over the course of 30 epochs showing that the Model was able to learn how to minimize the loss for the combined Loss Function. However, throughout this process, the val/f1 value remained at 0.000 which indicates that no meaningful Debris Detectors were detected; thus showing that the Model learned to minimize loss by predicting primarily non-Debris Detectors taking advantage of the Class Imbalance.

Run II (50 Epochs):

After training (epoch 0): ~0.569, After testing (epoch 49): ~0.346, Validation Loss after testing: ~0.443

Thus demonstrating more efficient learning than Run I, Run II's final training loss of 0.346 demonstrates a greater improvement than Run I on final training loss (compared with 0.569); the difference between training loss at ~0.346 vs validation at ~0.443 shows that, although some overfitting did exist, it remained at an acceptable level; and additionally, the val/f1 value of 0.300 further confirms that the model learned to identify debris pattern structures rather than merely producing all negative predictions.

5.2 Comparison with Existing Solutions

The results of this project are compared with relevant existing work to contextualize performance. The table below shows how this project compares with published approaches.

Approach	IoU	F1 Score	GPU Used	Dashboard
Kikaki et al. (2022)	0.30-0.50	~0.40	Yes	No
Mohammed et al. (2022)	~0.55	~0.60	Yes	No
This Project - Run 1	4.31%	8.26%	No (CPU)	No
This Project - Run 2	7.96%	14.74%	No (CPU)	Yes

In order to place the performance of the project within context and compare results collected during the research to work from published literature, results have been measured against existing projects (published benchmarks); these results are summarized in the table below. For example, the IoU of 7.96% is much lower than that of other projects trained on GPUs. This discrepancy is due to: 1) The fact that most epochs for training could only occur on CPU (due to time required each epoch); therefore only 50 epochs were trained in a realistically practical timeframe. 2) The image patch size (128x128) used in the current study is much smaller than those used in other publications (256x256); therefore, there was less spatial context available for the model to learn from). 3) As previously mentioned, there is less time available (approximately 17 minutes) for model learning during the training periods. Whereas, when using a GPU-accelerated system, model training can occur in less than one minute per epoch.

The most significant contribution of this project beyond previously published work includes the creation of the DebrisNet web dashboard; an easy-to-use, interactive interface for demonstrating the detection pipeline. None of the publications reviewed had an equivalent end-to-end demonstration tool for illustrating their actual detection pipeline. The complete detection pipeline (from raw satellite images collected from Sentinel-2 imagery through to web-based display of results) is a valuable contribution to the research and technology fields, as such a product makes the research more widely accessible to the public.

Future work employing GPU acceleration will greatly improve overall IoU metrics resulting in improved metric performance (IoU) to match or exceed published benchmarks; while still retaining the positive attributes attained from using the DebrisNet web dashboard.

5.3 Dashboard Performance Evaluation

Numerous aspects of performance were assessed for the DebrisNet web-based dashboard to evaluate the desktop and presentation value of the product during demonstration.

5.3.1 User Interface Performance

The following performance characteristics were demonstrated with the dashboard in the Google Chrome browser app running in Windows 11 environment:

All of the interactions tested within the dashboard provided very good responsiveness. The dashboard is entirely a client-side HTML file and, as such, it is not dependent on any server for operation, thus allowing it to be opened directly from any modern web browser without the need for installation or configuration.

5.3.2 Cross-Region Functionality

Two sets of satellite imagery were tested with the dashboard. They were:

All three tests were successful in showing that filename-based detection of regions is working correctly taken from the two regions that are currently supported.

5.3.3 Dashboard Limitations and Future Improvements

Although the detection pipeline can be demonstrated by the DebrisNet dashboard, a few limitations of the current implementation were discovered when assessing how well the dashboard performs: Currently, pre-computed detection results are used, rather than inferring detections live from uploaded images to the server. Thus the detection result will be already known before uploading the image. For proof that AIs are functioning as shown, a backend solution (like Flask or FastAPI) must be created to implement UNet++ inference on the uploaded image(s). Currently, the dashboard only supports two regions geographically and in order to expand, each new region would have to have a pre-computed detection mask created as well as updating the logic for filename detection for each of the new regions in the javascript code. Currently, the links for Hotspot Analysis, Model Insights and

Evaluation Metrics in the navigation pane are not yet functional; these pages should contain contents in the future to be of use to researchers and users of the dashboard.

5.4 Overall Project Evaluation

This section of this report presents a holistic assessment of the project as compared to its original goals defined in Chapter 1.

Objective	Status	Achievement
Develop UNet++ model for debris detection	☑ Achieved	Model trained, IoU 7.96%
Train using MARIDA with imbalance-aware strategy	☑ Achieved	Combined loss, pos_weight=50.0
Inference on Motagua River scene	☑ Achieved	2,712 debris pixels detected
Inference on South Africa scene	☑ Achieved	56 debris pixels detected
Develop DebrisNet web dashboard	☑ Achieved	Fully functional HTML dashboard
Evaluate using standard metrics	☑ Achieved	All 5 metrics reported
GPU-accelerated training	☐ Not achieved	AMD GPU not PyTorch compatible
IoU target of 0.30+	☐ Not achieved	Achieved 7.96% due to CPU limits

Five of the seven objectives were completely accomplished. The two objectives that were not achieved (GPU acceleration and higher IoU) could not be met because of hardware constraints, not from any methodological issues. The four components of the core detection pipeline (training methodology, inference, and visualization dashboard) worked as expected; together, these demonstrate that using UNet++ and Sentinel-2 imagery can successfully detect marine debris at the consumer-grade level, with plenty of opportunities to improve the model through GPU-accelerated training or an extended training time.

CHAPTER 06: CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

The automation pipeline for detecting marine plastic debris from Sentinel-2 imagery has been successfully developed, implemented and validated through the use of deep learning techniques. The detection system utilises a UNET++ segmentation model with a ResNet34 backbone structure, which allows for the detection of marine plastic debris in Sentinel-2 imagery using an imbalance aware loss function trained on the MARIDA benchmark dataset. Additionally, the entire process was conducted on a consumer level processing system (AMD Ryzen 5 5625U, 15.3GB RAM, no GPU) and has demonstrated that the methodology can be used outside of traditional research environments.

The principal results of this research study are summarized below:

1. **Successful CPU-Based Training:** Successful CPU training of UNet++ was able to produce 50 epochs on CPU with a 99.9% final test accuracy, F1 Score of 14.74%, Precision of 9.44%, Recall of 33.60%, and IoU of 7.96%. Both increasing the value for pos_weight to 50.0 and lowering the learning rate used in training to 5e-5 in the 2nd training run resulted in an increase in performance compared to baseline training runs with val/f1 increasing from .000 –.300 fewval during validation stages.
2. **Effective Inference on Real Satellite Data:** The effectiveness of UNet++ to be able to successfully provide inference results for the Motagua River Plume (18 September 2020) produced clear linear debris trail with 2,712 detections for pixels detected that make sense based on debris patterns observed for this highly polluted river basin. There were also 56 detections on the Durban, South Africa 29 April 2019 sites within hypothetical water areas located in plausible port areas indicating this model's ability to function in other geographical areas.
3. **DebrisNet Web Dashboard:** A professional web-based dashboard (called DebrisNet) utilizing HTML5 and CSS3 and JavaScript was created. The dashboard provides interactivity for uploading satellite images, visualizing detected debris, and presenting results with regionally specific updates allowing for users who may have minimal technical knowledge to

access detection technologies for demonstration purposes, as well as in terms of educational purposes.

4. **Complete End-to-End Pipeline:** A professional Interactive Web Dashboard (DebrisNet) was developed using HTML5, CSS3, and JavaScript. Through the use of an interactive user interface, DebrisNet allows users to upload satellite images and visualize detected debris using the same technology used for training, educational, and demonstration purposes with the intention of providing non-technical users and/or users who have limited technical expertise an interface to demonstrate the use of detection technologies in their respective geographic regions.
5. **Practical Problem Solving:** During the development of this image classification system, a number of engineering challenges were solved that included how to manage disk space, optimize CPU-only training and to prevent training interruptions during running time and how to convert satellite images into a browser-friendly format.

6.1.1 LIMITATIONS

- **IoU Performance Gap:** The IoU reached was 7.96%, which is far lower than the average GPU trained benchmarks of 0.30-0.55, due to using CPU only to train the system, which only allowed for 50 epochs of practical training within reasonable time limits as opposed to 100+ epochs for various benchmarks.
- **No GPU Acceleration:** Because AMD does not currently support AMD GPUs with PyTorch for Windows development, all training on the AMD Radeon GPU could only be done at a maximum speed of around 0.65 every second and maximally 24 per day.
- **Spatial Resolution Constraints:** Detection is mostly limited due to the 10m spatial resolution of Sentinel-2 imagery; thus individual small objects have no chance of being detected through this data. Microplastic fragments are unable to be detected individually through this source either.
- **Dashboard Simulation Only:** The existing dashboard for DebrisNet presently generates detection results in advance of image upload and only serves the purpose of showing how it can be used.

- **Limited Geographic Coverage:** All inference that was provided was made via pre-generated detection results and therefore had restricted practical application; it demonstrated the effective detection of debris for only two geographic regions (Arctic & Tropical), and validation of model generalization across various geographical marine environments (open ocean, etc.) has not been adequately validated.

6.1.2 CONTRIBUTIONS TO THE FIELD

- Created a full, modular, and reproducible end-to-end pipeline to convert raw Sentinel 2 imagery into debris probability maps and web-based visualization outputs.
- Showed that UNet++ can be trained without using dedicated GPUs to detect ocean debris using only CPUs, thus providing all researchers with access to this technology.
- Developed DebrisNet as an open/demonstration tool that can be used/extended by researchers, governments and environmental practitioners.
- Successfully utilized trained model to produce scientifically reasonable debris detection in 2 actual satellite scenes, both of which originate from the Motagua River in Guatemala and Durban, South Africa.
- Documented engineering best practices to address issues like: managing disk space, limitations imposed by AMD GPUs and avoiding interruptions from training users which can be benefitted from by other researchers who are doing work in resource-limited environments.

6.2 FUTURE SCOPE

● **GPU-Accelerated Training:** If we were to deploy our sources of machine learning training through a cloud-based GPU service (e.g. Google Colab, AWS EC2, Kaggle Notebooks), this would allow for training for 100+ epochs with batch sizes of 16-32 and would allow for using larger image patches of 256x256; this structure is predicted to increase the IoU significantly to published benchmarks of 0.30-0.55 while using the same architecture of the pipeline..

● **Live Detection Backend:** By integrating the inference pipeline with a Flask or FastAPI Python backend server, the ongoing work of developing a DebrisNet dashboard will shift from a demonstration of the dashboard to a complete functioning application that enables real-time detection of uploaded images rather than producing results from a set of pre-generated images.

- **Additional Training Data:** The model will benefit from generalising across many marine habitats by producing or obtain new data sources such as FloatingObjects, PLASTIC-MARIDA, and synthetic spectral datasets that can be added to the various marine habitat types we are modelling in this study (i.e., open ocean, Arctic, and tropical coastal/estuary).
- **Hotspot Analysis Page:** The second priority (or third priority) is to develop the DebrisNet Hotspot Analysis navigation page on the DebrisNet dashboard through development in Leaflet.js. This page will allow for the presentation of georeferenced maps of debris accumulation hotspots. The Hotspot Analysis will provide NGOs with information on where to target for clean-up operations.
- **Model Insights Page:** Incorporating training curves, confusion matrices, precision-recall curves, and performance metrics into the Model Insights page will enhance the dashboard's educational and research value.
- **Multi-Sensor Fusion:** Combining optical Sentinel-2 data and Synthetic Aperture Radar (SAR) data obtained from the Sentinel-1 satellite will enable the detection of objects in the presence of cloud cover that would not be visible using optical imagery alone, resulting in a large increase in the temporal coverage of the monitoring system.
- **Real-Time Monitoring:** Future research will concentrate on improving the efficiency of the model for near real-time performance, using methods such as model pruning, quantisation, and lightweight architectures. The use of live data from Sentinel-2 as an input into the model will allow for faster detection of and notification to emerging plastic accumulation areas.
- **Extended Geographic Coverage:** In addition to the above, the pipeline will also be tested in other areas of the world, including the Mediterranean Sea, Southeast Asia's coastal areas, and the Arctic Ocean, to prove the model's ability to generalise and provide global debris maps.

REFERENCES

- [1] A. Kikaki et al., "MARIDA: A benchmark for Marine Debris detection from Sentinel-2 remote sensing data," *PLOS ONE*, vol. 17, no. 5, pp. e0268129, May 2022.
- [2] A. Mohammed et al., "ResAttUNet: Detecting Marine Debris using an Attention Activated Residual U-Net," *arXiv preprint arXiv:2206.09791*, 2022.
- [3] M. Rußwurm, S. Jilla Venkatesa, D. Tuia, et al., "Large-scale detection of marine debris in coastal areas with Sentinel-2," *iScience*, vol. 26, no. 7, 2023.
- [4] B. M. de Barros, et al., "Spectral Analysis of Marine Debris in Simulated and Observed Sentinel-2/MSI Images using Unsupervised Classification," *arXiv preprint arXiv:2301.04567*, 2023.
- [5] V. Nivedita et al., "Plastic debris detection along coastal waters using machine learning," *Environmental Advances*, vol. 5, no. 2, pp. 100321, Jan. 2024.
- [6] M. O. Edeh et al., "A novel deep learning model for predicting marine plastic pollution," *PeerJ Computer Science*, vol. 7, pp. e562, Apr. 2024.
- [7] S. Dhanya, R. Gudan, and K. G. Kirthika, "Marine Plastic Detection Using Deep Learning," *International Journal of Research in Engineering and Science (IJRES)*, vol. 10, no. 7, pp. 45– 52, Jul. 2022.
- [8] S. Sannigrahi et al., "Development of automated marine floating plastic detection using Sentinel satellite data," *PubMed*, 2022.
- [9] H. Booth, W. Ma, O. Karakus, "High-precision density mapping of marine debris and floating plastics via satellite imagery," *Nature Sustainability*, vol. 6, pp. 214– 225, Feb. 2023.
- [10] A. Danilov, E. Serdiukova, "Review of Methods for Automatic Plastic Detection in Water Areas Using Satellite Images and Machine Learning," *MDPI Remote Sensing*, vol. 16, no. 2, pp. 112– 130, 2024.
- [11] N. Prakash et al., "AI-enhanced real-time monitoring of marine pollution: A comprehensive review," *Marine Pollution Bulletin*, vol. 182, pp. 113927, Mar. 2025.
- [12] S. Sannigrahi, B. Basu, A. S. Basu, F. Pilla, "Detection of Marine Floating Plastic using Sentinel-2 imagery and machine learning models," *arXiv preprint arXiv:2106.03562*, 2021.
- [13] H. Booth, W. Ma, O. Karakus, "High-precision Density Mapping of Marine Debris and Floating Plastics via Satellite Imagery," *arXiv preprint arXiv:2203.02991*, 2022.

- [14] M. Srinivasa Rao, "Hybrid DL Approach for Marine Debris Detection Using UNet with ResNext50," *Journal of Applied Science and Technology Trends*, vol. 15, no. 4, 2025.
- [15] B. M. Bhanumathi, "Marine Plastic Detection Using Deep Learning," *Advances in Parallel Computing*, vol. 42, pp. 321–330, 2022.
- [16] Y. Wei et al., "Marine Debris Classification with Improved YOLO v8," *MDPI Remote Sensing*, vol. 14, no. 22, 2023.
- [17] ESA, "Plastic Litter Detection with Deep Learning from Satellites," *ESA R&D Project Report*, 2021.
- [18] M. P. Raju et al., "ML-based Litter Detection along Indian Coast," *Frontiers in Marine Science*, vol. 12, pp. 998123, Apr. 2025.
- [19] M. Balsi et al., "Plastic Detection with Hyperspectral Aerial Remote Sensing," *Remote Sensing Letters*, vol. 16, no. 3, pp. 233–244, 2025.
- [20] J. Pretorius et al., "Remote Sensing of Marine Plastics in South Africa," *ISPRS Annals of Photogrammetry*, vol. 10, no. 1, pp. 127–139, 2025.
- [21] D. P. Kingma, J. Ba, "Adam: A Method for Stochastic Optimization," *Proc. International Conference on Learning Representations (ICLR)*, 2015.
- [22] O. Ronneberger, P. Fischer, T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *MICCAI 2015*, pp. 234–241.
- [23] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," *USENIX Conference on Operating Systems Design and Implementation (OSDI)*, 2016.
- [24] Z. Zhou, M. M. Rahman Siddiquee, N. Tajbakhsh, J. Liang, "UNet++: A Nested U-Net Architecture for Medical Image Segmentation," *Deep Learning in Medical Image Analysis*, Springer, 2018.
- [25] L. Biermann, D. Clewley, V. Martinez-Vicente, K. Topouzelis, "Finding Plastic Patches in Coastal Waters using Optical Satellite Data," *Scientific Reports*, vol. 10, no. 1, pp. 6094, 2020.

JAYPEE UNIVERSITY OF INFORMATION TECHNOLOGY, WAKNAGHAT**PLAGIARISM VERIFICATION REPORT**

Date:

Type of Document (Tick): ☐ PhD Thesis ☐ M.Tech Dissertation/ Report ☐ B.Tech Project Report ☐ Paper

Name: _____ Department: _____ Enrolment No _____

Contact No. _____ E-mail. _____

Name of the Supervisor: _____

Title of the Thesis/Dissertation/Project Report/Paper (In Capital letters): _____

UNDERTAKING

I undertake that I am aware of the plagiarism related norms/ regulations, if I found guilty of any plagiarism and copyright violations in the above thesis/report even after award of degree, the University reserves the rights to withdraw/revoke my degree/report. Kindly allow me to avail Plagiarism verification report for the document mentioned above.

Complete Thesis/Report Pages Detail:

- Total No. of Pages =
- Total No. of Preliminary pages =
- Total No. of pages accommodate bibliography/references =

(Signature of Student)

FOR DEPARTMENT USE

We have checked the thesis/report as per norms and found **Similarity Index** at(%). Therefore, we are forwarding the complete thesis/report for final plagiarism check. The plagiarism verification report may be handed over to the candidate.

(Signature of Guide/Supervisor)

Signature of HOD

FOR LRC USE

The above document was scanned for plagiarism check. The outcome of the same is reported below:

Copy Received on	Excluded	Similarity Index (%)	Generated Plagiarism Report Details (Title, Abstract & Chapters)	
	• All Preliminary Pages • Bibliography/Images/Quotes • 14 Words String		Word Counts	
Report Generated on			Character Counts	
		Submission ID	Total Pages Scanned	
			File Size	

Checked by
Name & Signature

Librarian

Please send your complete thesis/report in (PDF) with Title Page, Abstract and Chapters in (Word File) through the supervisor at plagcheck.juit@gmail.com

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI-generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



12% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **139 Not Cited or Quoted 10%**
Matches with neither in-text citation nor quotation marks.
-  **5 Missing Quotations 0%**
Matches that are still very similar to source material.
-  **21 Missing Citation 1%**
Matches that have quotation marks, but no in-text citation.
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks.

Top Sources

- 9%  Internet sources
- 7%  Publications
- 7%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **139 Not Cited or Quoted 10%**
Matches with neither in-text citation nor quotation marks
- **5 Missing Quotations 0%**
Matches that are still very similar to source material
- **21 Missing Citation 1%**
Matches that have quotation marks, but no in-text citation
- **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 9% Internet sources
- 7% Publications
- 7% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	ir.juit.ac.in:8080	2%
2	Student papers	Jaypee University of Information Technology	<1%
3	Student papers	University of San Diego	<1%
4	Internet	eartharxiv.org	<1%
5	Student papers	University of Warwick	<1%
6	Student papers	University of Strathclyde	<1%
7	Internet	proceedings.stis.ac.id	<1%
8	Internet	cdn.files.pg.edu.pl	<1%
9	Internet	www.mdpi.com	<1%
10	Internet	lume.ufrgs.br	<1%

11	Publication	Dimitris V. Politikos, Argyro Adamopoulou, George Petasis, Francois Galgani. "Usi...	<1%
12	Student papers	City University of Hong Kong	<1%
13	Internet	arxiv.org	<1%
14	Student papers	Indian School of Business	<1%
15	Publication	Soufyane Bouchelaghem, Lahcene Mamen, Marco Balsi, Ahmed Tibermacine, Mo...	<1%
16	Student papers	University of Waikato	<1%
17	Internet	www.ir.juit.ac.in:8080	<1%
18	Internet	etheses.whiterose.ac.uk	<1%
19	Student papers	Johnson County Community College	<1%
20	Student papers	IIIT Kottayam	<1%
21	Internet	thelinuxcode.com	<1%
22	Student papers	Blackboard Learn Production	<1%
23	Student papers	Pennsylvania State System of Higher Education	<1%
24	Student papers	TU Delft	<1%